Final Project Study

Zeynep TUTAR

```r
library(readxl)  #for reading excel
library(lmtest)  #for the use of DW test, autocorrelation check
library(forecast)  #for linear reg for time series
library(DIMORA)  #for innovation dif models (GBM, Bass Model etc.)
library(naniar)
library(ggplot2)
library(corrplot)
library(car)
library(dplyr)
library(tidyr)
library(prophet)
library(MLmetrics)
library(knitr)
```

1) Data Exploration & Preprocessing

Load and Inspect the Dataset

```r
ott_cinema_monthly <- read_excel("ott_cinema_dataset.xlsx", sheet = "Monthly")
str(ott_cinema_monthly)
```

```
tibble [216 x 9] (S3: tbl_df/tbl/data.frame)
 $ Time                    : chr [1:216] "2006-01" "2006-02" "2006-03" "2006-04" ...
 $ cinema_admissions       : num [1:216] 13700979 8207270 10102967 10872037 9408449 ...
 $ cinema_screenings       : num [1:216] 120005 102136 109564 110636 103219 ...
 $ cinema_audience_spending: num [1:216] 89522221 50597282 61407884 68866213 58029889 ...
 $ cinema_avg_spending_pp  : num [1:216] 6.53 6.16 6.08 6.33 6.17 ...
 $ gooogle_trend_ott       : num [1:216] 0 0 0 0 0 0 0 0 0 0 ...
 $ cpi_all                 : num [1:216] 0.2 0.2 0.2 0.3 0.2 0.2 0.4 0.2 -0.1 -0.2 ...
 $ cpi_culture             : num [1:216] 0.6 -0.2 0.2 -0.2 -0.1 0.1 0.4 0.7 0.2 -0.6 ...
 $ covid_cases             : num [1:216] 0 0 0 0 0 0 0 0 0 0 ...
```

```r
ott_cinema_annual <- read_excel("ott_cinema_dataset.xlsx", sheet = "Annual")
str(ott_cinema_annual)
```

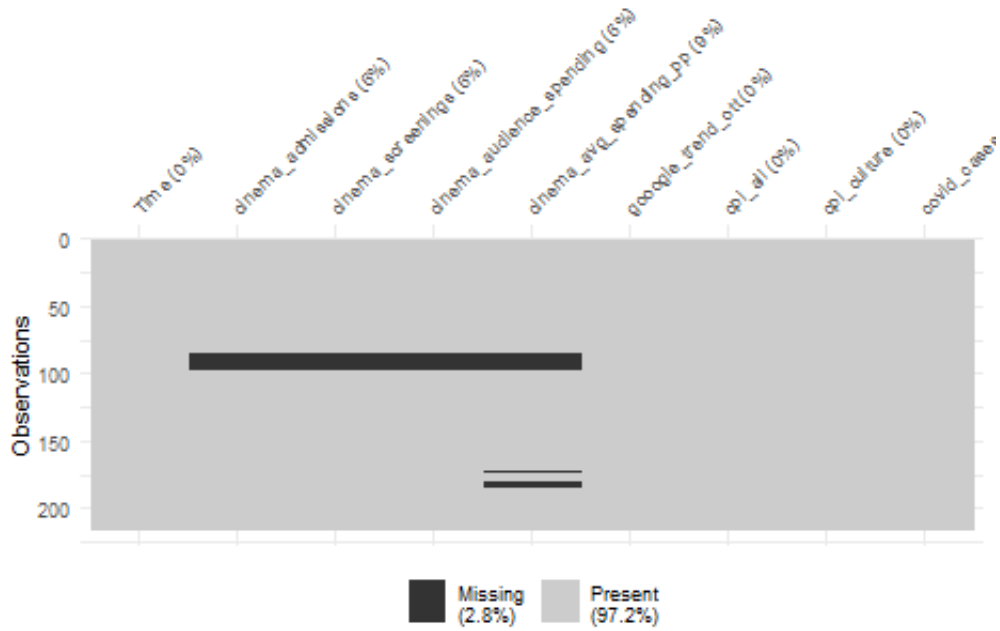
```
tibble [18 x 4] (S3: tbl_df/tbl/data.frame)
 $ Year               : num [1:18] 2006 2007 2008 2009 2010 ...
 $ ott_subscriptions_M: num [1:18] 0 0 0 0 0 0 0 0 0.3 0.8 ...
 $ ott_spending_M_EUR : num [1:18] 0 0 0.6 0.7 1.4 9.5 21.9 21.1 39 64.1 ...
 $ cinema_admissions  : num [1:18] 1.06e+08 1.17e+08 1.12e+08 1.11e+08 1.23e+08 ...
```

```r
# ott_cinema_monthly$Time=as.Date(paste0(ott_cinema_monthly$Time,
# '-01'), format = '%Y-%m-%d') ott_cinema_annual$Year <-
# as.Date(paste0(ott_cinema_annual$Year, '-01-01'), format='%Y-%m-%d')
```

```r
# Check for missing values
ott_cinema_monthly[apply(is.na(ott_cinema_monthly), 1, any), ]
```

```
# A tibble: 19 x 9
   Time    cinema_admissions cinema_screenings cinema_audience_spending
   <chr>               <dbl>             <dbl>                     <dbl>
 1 2013-01                NA                NA                        NA
 2 2013-02                NA                NA                        NA
 3 2013-03                NA                NA                        NA
 4 2013-04                NA                NA                        NA
 5 2013-05                NA                NA                        NA
 6 2013-06                NA                NA                        NA
 7 2013-07                NA                NA                        NA
 8 2013-08                NA                NA                        NA
 9 2013-09                NA                NA                        NA
10 2013-10                NA                NA                        NA
11 2013-11                NA                NA                        NA
12 2013-12                NA                NA                        NA
13 2020-04                 0                 0                       326.
14 2020-05                 0                 0                       198
15 2020-11                 0                 0                      4824.
16 2020-12                 0                 0                     56332
17 2021-01                 0                 0                     27483
18 2021-02                 0                 0                     14354.
19 2021-03                 0                 0                      2165.
# i 5 more variables: cinema_avg_spending_pp <dbl>, gooogle_trend_ott <dbl>,
#   cpi_all <dbl>, cpi_culture <dbl>, covid_cases <dbl>
```

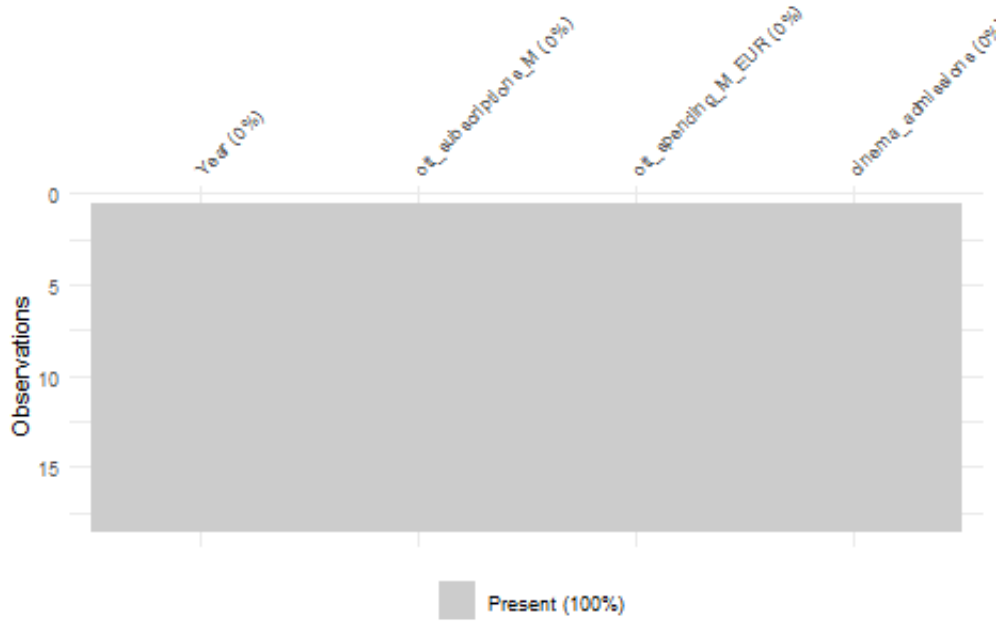
```r
vis_miss(ott_cinema_monthly)
```




```r
ott_cinema_annual[apply(is.na(ott_cinema_annual), 1, any), ]
```

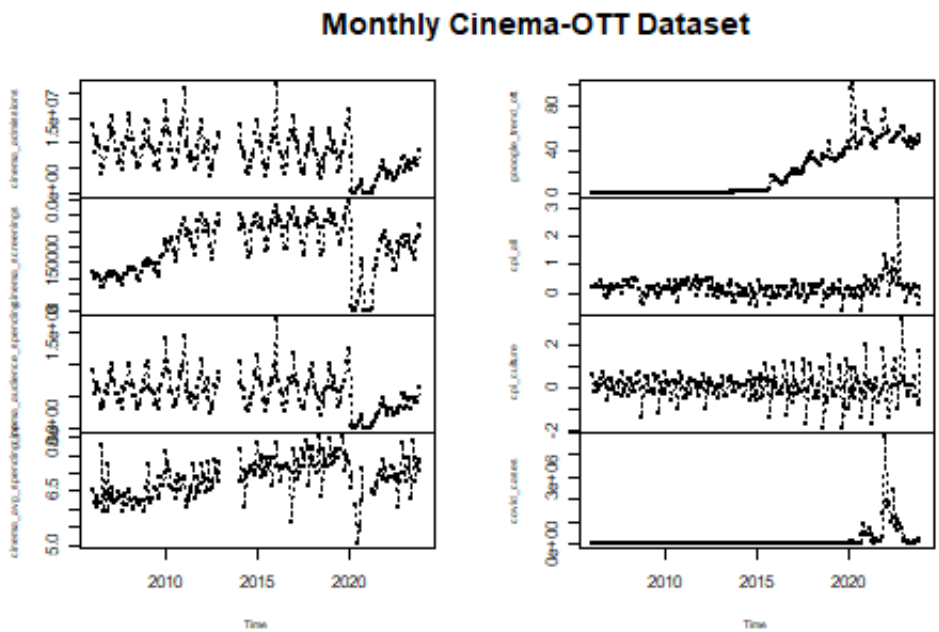
```
# A tibble: 0 x 4
# i 4 variables: Year <dbl>, ott_subscriptions_M <dbl>,
#   ott_spending_M_EUR <dbl>, cinema_admissions <dbl>
```

```r
vis_miss(ott_cinema_annual)
```

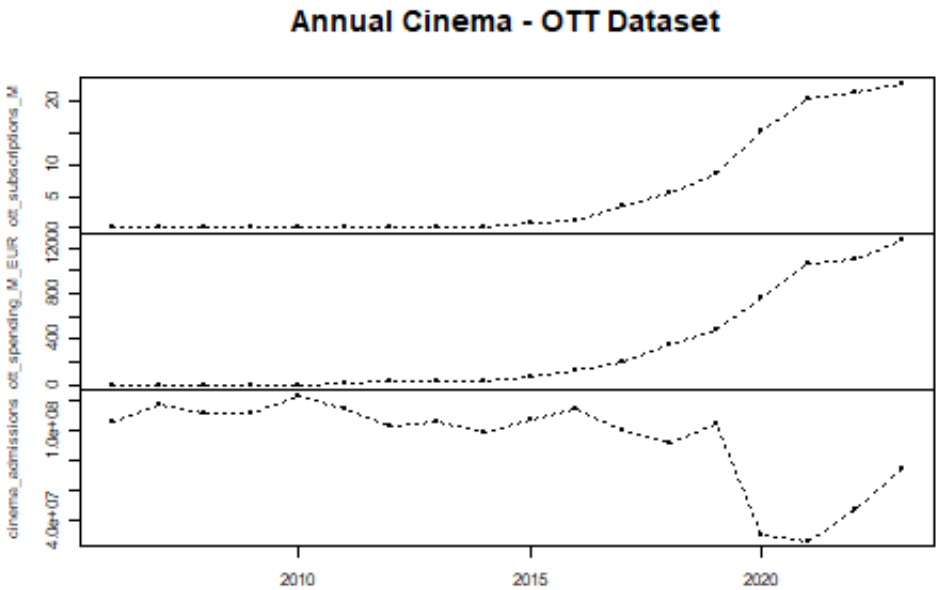


```r
ott_cinema_monthly.ts <- ts(ott_cinema_monthly[, -1], start = c(2006, 1),
    end = c(2023, 12), frequency = 12)
ott_cinema_annual.ts <- ts(ott_cinema_annual[, -1], start = c(2006), end = c(2023),
    frequency = 1)
```

```r
plot(ott_cinema_monthly.ts, main = "Monthly Cinema-OTT Dataset", cex.lab = 0.45,
    type = "o", pch = 16, lty = 3, cex = 0.6)
```




```r
plot(ott_cinema_annual.ts, main = "Annual Cinema - OTT Dataset", cex.lab = 0.6,
    type = "o", pch = 16, lty = 3, cex = 0.6, xlab = "")
```



Handle Missing Values

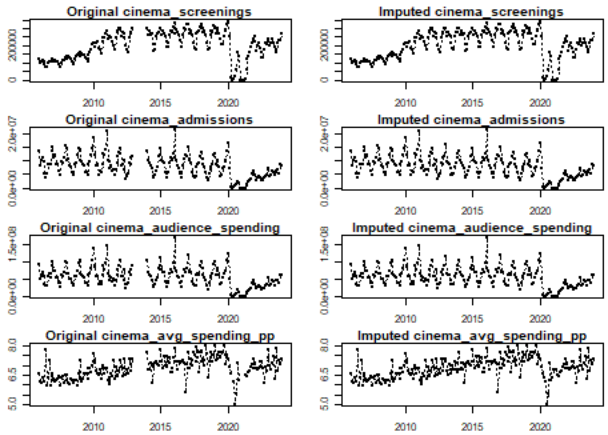
Handle Monthly Dataset Missing Values

```
# Define the total annual values for 2013 for each column
total_2013 <- list(cinema_screenings = 3014642, cinema_admissions = 106124433,
    cinema_audience_spending = 732299026.82)

# Define the columns to impute
columns_to_impute <- c("cinema_screenings", "cinema_admissions", "cinema_audience_spending",
    "cinema_avg_spending_pp")

# Create a copy of the time series object for imputation
cinema_imputed <- ott_cinema_monthly.ts

# Filter data to include only 2011, 2012, 2014, 2015 for monthly
# averages
selected_years <- (time(ott_cinema_monthly.ts) >= 2011 & time(ott_cinema_monthly.ts) <
    2013) | (time(ott_cinema_monthly.ts) >= 2014 & time(ott_cinema_monthly.ts) <
    2016)
```

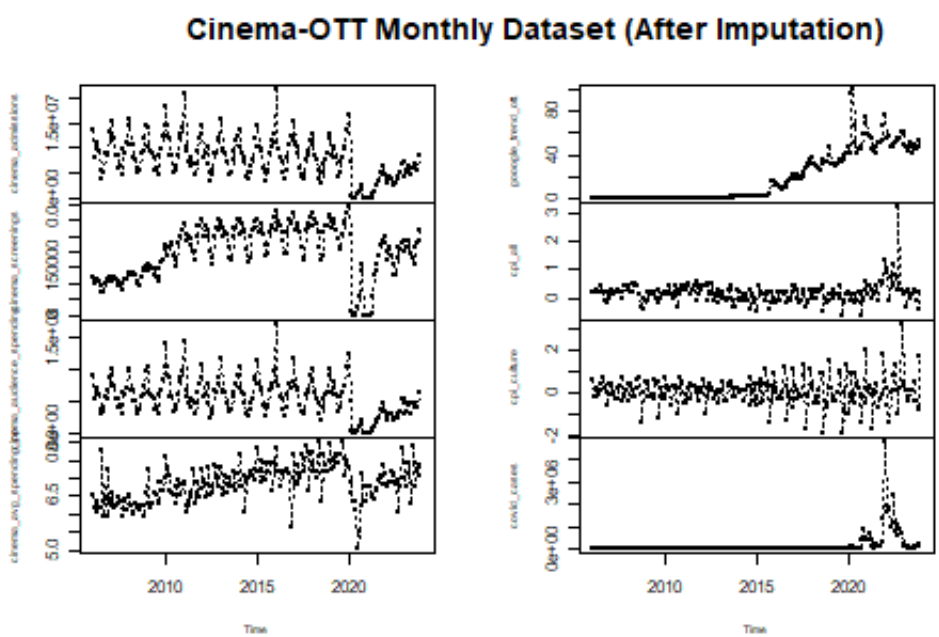
```
# Loop through each column to impute missing values
for (col in columns_to_impute) {
    # Extract the column data
    column_data <- as.numeric(ott_cinema_monthly.ts[, col])

    if (col == "cinema_avg_spending_pp") {
        # Calculate avg_spending_pp using the imputed
        # audience_spending/audience_admissions
```

```r
        imputed_2013 <- cinema_imputed[, "cinema_audience_spending"]/cinema_imputed[,
            "cinema_admissions"]

        # Replace the missing 2013 data with the calculated values
        column_data[time(ott_cinema_monthly.ts) >= 2013 & time(ott_cinema_monthly.ts) <
            2014] <- imputed_2013[time(ott_cinema_monthly.ts) >= 2013 & time(ott_cinema_monthly.ts) <
            2014]

        # Handle missing values in 2020 and 2021 (due to COVID-19
        # impact)
        column_data[time(ott_cinema_monthly.ts) >= 2020 & time(ott_cinema_monthly.ts) <
            2021 & is.na(column_data)] <- 6.35  # Average for 2020
        column_data[time(ott_cinema_monthly.ts) >= 2021 & time(ott_cinema_monthly.ts) <
            2022 & is.na(column_data)] <- 6.76  # Average for 2021
    } else {
        # Handle cumulative features using selected years for
        # proportions
        monthly_proportions <- colMeans(matrix(column_data[selected_years],
            ncol = 12, byrow = TRUE), na.rm = TRUE)
        # Normalize to proportions
        monthly_proportions <- monthly_proportions/sum(monthly_proportions)
        imputed_2013 <- monthly_proportions * total_2013[[col]]

        # Replace the missing 2013 data with the imputed values
        column_data[time(ott_cinema_monthly.ts) >= 2013 & time(ott_cinema_monthly.ts) <
            2014] <- imputed_2013
    }

    # Update the imputed data in the time series
    cinema_imputed[, col] <- column_data
}
```

```r
# Plot the original and imputed data for comparison
par(mfrow = c(4, 2), mar = c(2, 3, 1.5, 1))
for (col in columns_to_impute) {
    # par(mfrow = c(2, 1), mar = c(4, 4, 2, 1))

    # Plot the original data
    plot(ott_cinema_monthly.ts[, col], type = "o", pch = 16, lty = 3, cex = 0.4,
        main = paste("Original", col), xlab = "", ylab = "")

    # Plot the imputed data
    plot(cinema_imputed[, col], type = "o", pch = 16, lty = 3, cex = 0.4,
        main = paste("Imputed", col), xlab = "", ylab = "")
}

par(mfrow = c(1, 1))
```




```
ott_cinema_monthly.ts <- cinema_imputed
ott_cinema_monthly <- as.data.frame(cinema_imputed)

# Final plot of the imputed dataset
plot(ott_cinema_monthly.ts, main = "Cinema-OTT Monthly Dataset (After Imputation)",
    cex.lab = 0.45, type = "o", pch = 16, lty = 3, cex = 0.6)
```



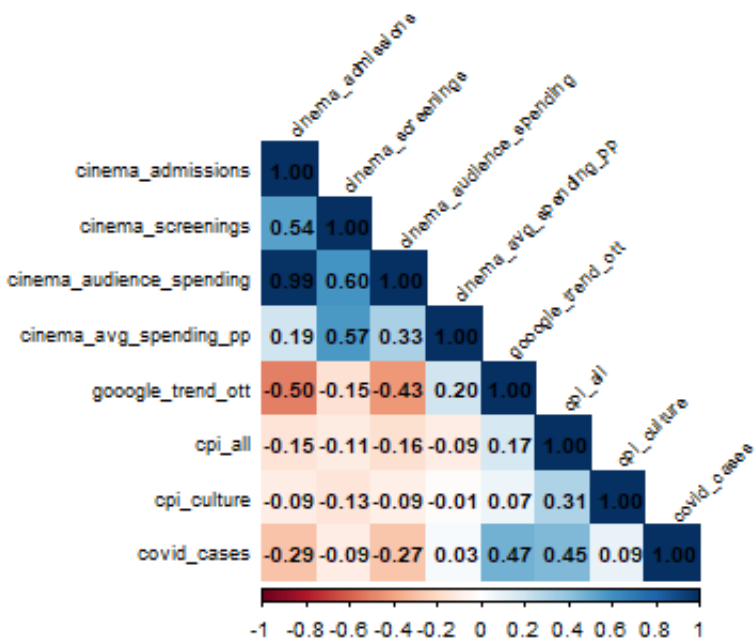
Collinearity Check

Correlation Matrix for Monthly Data

```
# Correlation Matrix for Monthly Data
ott_cinema_corr <- as.data.frame(ott_cinema_monthly.ts)
```

```r
# Compute correlation matrix
cor_matrix_monthly <- cor(ott_cinema_corr, use = "complete.obs")
```

```r
# default_margins <- par('mar') par(mar = c(5, 5, 4, 2) + 0.1)

# Visualize correlation matrix
corrplot(cor_matrix_monthly, method = "color", type = "lower", tl.col = "black",
    tl.srt = 45, addCoef.col = "black", number.cex = 0.8, tl.cex = 0.75)
```




```r
# title(main = 'Correlation Matrix: Monthly Data', line = 2, cex.main =
# 1.2) par(mar = default_margins)
```

VIF Analysis for Monthly Data

```r
# Run VIF analysis Regression model with all predictors
vif_model.monthly <- lm(ott_cinema_monthly$cinema_admissions ~ . - ott_cinema_monthly$cinema_admissions
    data = ott_cinema_monthly[, -1])
# Compute VIF
VIF.values <- vif(vif_model.monthly)
kable(as.data.frame(VIF.values), caption = "VIF Table for Monthly Data")
```

Table 1: VIF Table for Monthly Data

	VIF.values
cinema_screenings	2.126995
cinema_audience_spending	1.976895

	VIF.values
cinema_avg_spending_pp	1.771816
gooogle_trend_ott	1.746757
cpi_all	1.389224
cpi_culture	1.130118
covid_cases	1.578432

Check correlation & trends between OTT Data (Annual) and Google Trends OTT (Monthly)

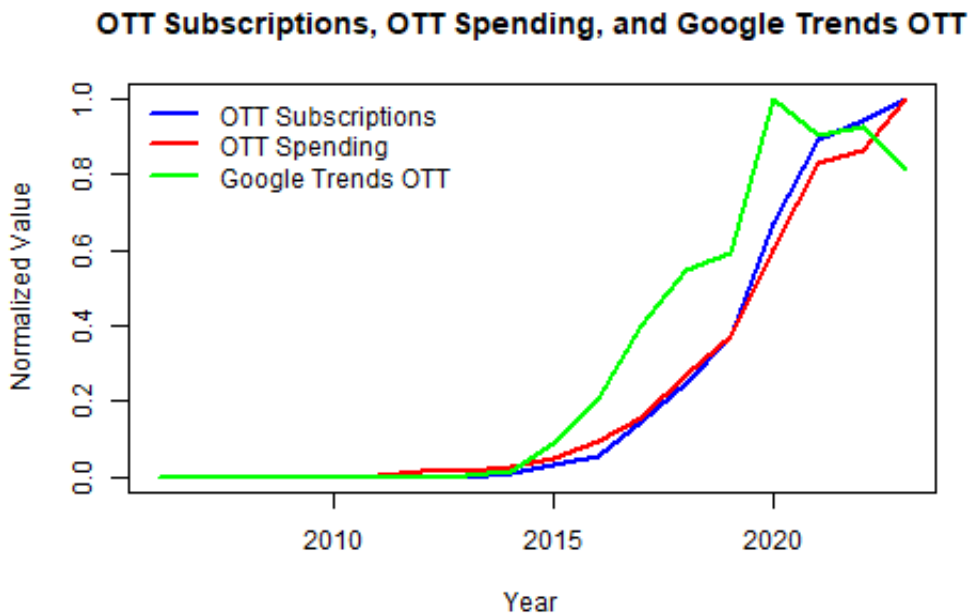
```
ott_cinema_monthly$Year <- rep(2006:2023, each = 12)  # Create a Year column

# Aggregate Google Trends OTT monthly data to annual (sum)
google_trends_annual <- ott_cinema_monthly %>%
    group_by(Year) %>%
    summarise(google_trend_ott = sum(gooogle_trend_ott, na.rm = TRUE))

# Merge with annual dataset
merged_ott_data <- merge(ott_cinema_annual[, c(1:3)], google_trends_annual,
    by.x = "Year", by.y = "Year", all.x = TRUE)
```

```
# Compute Correlation Matrix
cor_matrix <- cor(merged_ott_data[, -1], use = "complete.obs")
cor_matrix
```

```
                    ott_subscriptions_M ott_spending_M_EUR google_trend_ott
ott_subscriptions_M           1.0000000          0.9980075        0.9379704
ott_spending_M_EUR            0.9980075          1.0000000        0.9344162
google_trend_ott              0.9379704          0.9344162        1.0000000
```

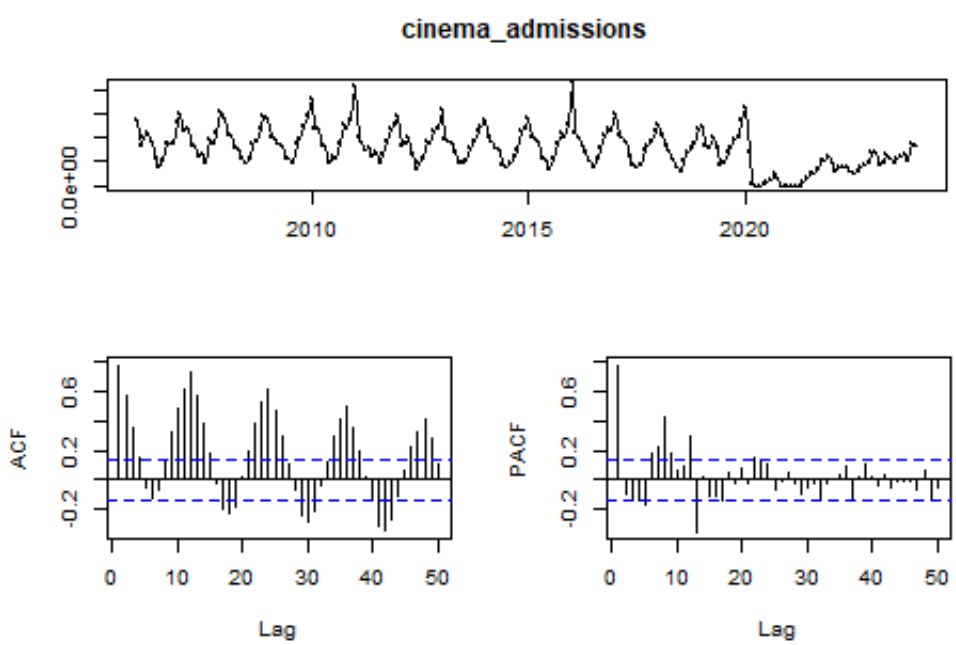
```
min_max_scale <- function(x) {
    return((x - min(x, na.rm = TRUE))/(max(x, na.rm = TRUE) - min(x, na.rm = TRUE)))
}

# Apply Min-Max Scaling to the relevant columns
merged_ott_data$ott_subscriptions_scaled <- min_max_scale(merged_ott_data$ott_subscriptions_M)

merged_ott_data$ott_spending_scaled <- min_max_scale(merged_ott_data$ott_spending_M_EUR)

merged_ott_data$google_trend_ott_scaled <- min_max_scale(merged_ott_data$google_trend_ott)
```

```
# Compare trends
plot(merged_ott_data$Year, merged_ott_data$ott_subscriptions_scaled, type = "l",
    col = "blue", lwd = 2, xlab = "Year", ylab = "Normalized Value", main = "OTT Subscriptions, OTT Spe
lines(merged_ott_data$Year, merged_ott_data$ott_spending_scaled, col = "red",
    lwd = 2)
lines(merged_ott_data$Year, merged_ott_data$google_trend_ott_scaled, col = "green",
    lwd = 2)
legend("topleft", legend = c("OTT Subscriptions", "OTT Spending", "Google Trends OTT"),
    col = c("blue", "red", "green"), lwd = 2, bty = "n")
```

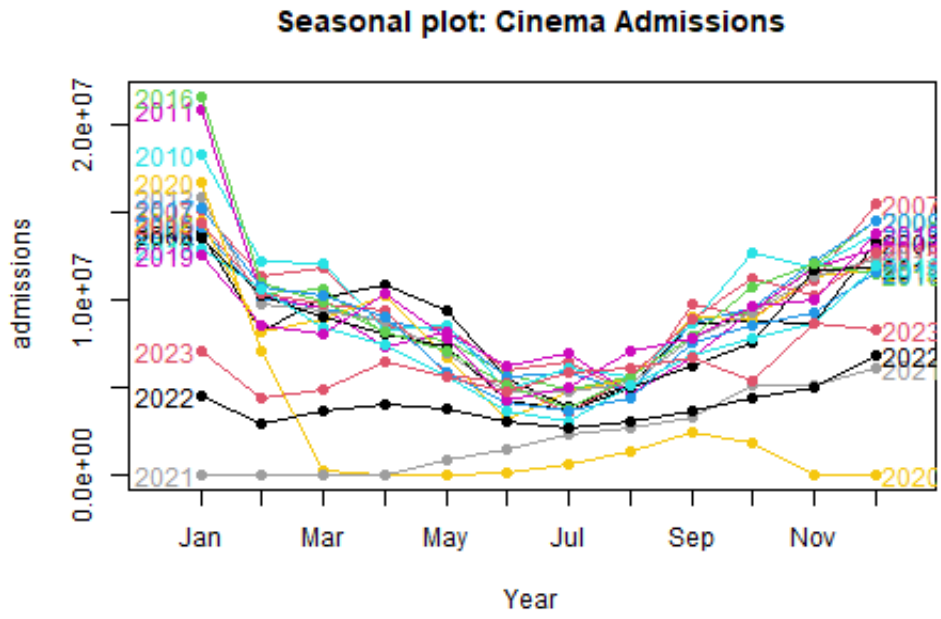


Time Series Decomposition

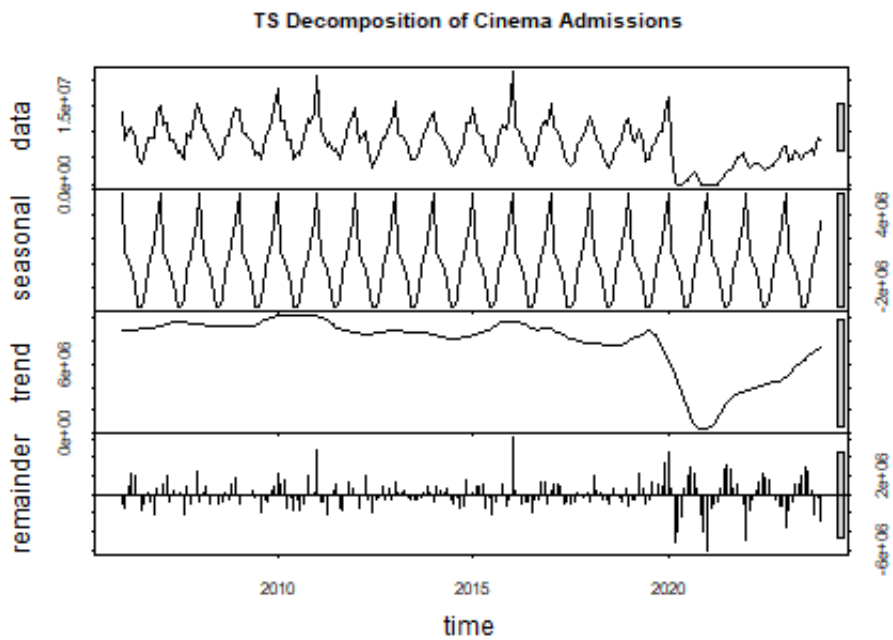
```
# Extract Cinema Admissions time series
cinema_admissions <- ott_cinema_monthly.ts[, "cinema_admissions"]

# a general view of the data
tsdisplay(cinema_admissions, lag.max = 50)
```



```r
seasonplot(cinema_admissions, ylab = "admissions", xlab = "Year", main = "Seasonal plot: Cinema Admissi
    year.labels = T, year.labels.left = T, col = 1:20, pch = 19)
```




```r
# STL decomposition (Seasonal + Trend + Residual)
admissions_decomposed <- stl(cinema_admissions, s.window = "periodic")

# Plot the decomposed components
plot(admissions_decomposed, main = "TS Decomposition of Cinema Admissions",
    cex.lab = 0.45, pch = 16, lty = 1, cex = 0.6)
```



2) Time Series Forecasting

Linear Regression

```
cinema_screening <- ott_cinema_monthly.ts[, 2]
cinema_spending <- ott_cinema_monthly.ts[, 3]
avg_cinema_spending_pp <- ott_cinema_monthly.ts[, 4]
google_trend_ott <- ott_cinema_monthly.ts[, 5]
cpi_all <- ott_cinema_monthly.ts[, 6]
cpi_culture <- ott_cinema_monthly.ts[, 7]
covid_cases <- ott_cinema_monthly.ts[, 8]

metrics <- c("Model", "AIC", "DW_Test", "Ljung_Box_pval", "R2", "MAE", "RMSE")
model_performance <- data.frame(matrix(ncol = length(metrics), nrow = 0))
colnames(model_performance) <- metrics
```

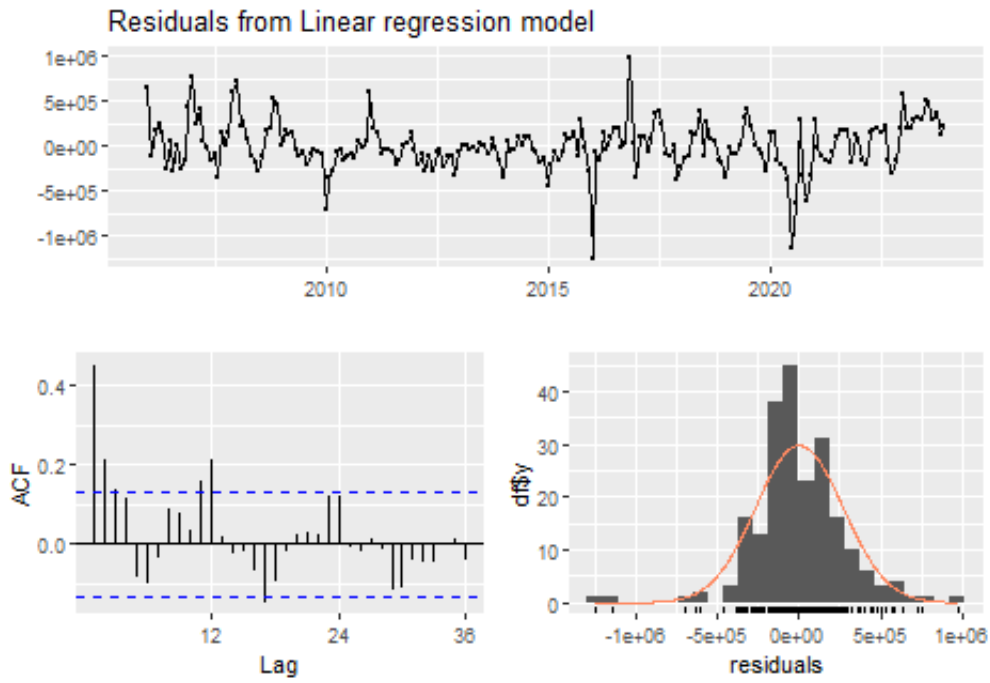
Multiple Linear Regression with Time Series

```
# Fit Initial Full Model
full_tslm <- tslm(cinema_admissions ~ cinema_screening + cinema_spending +
    avg_cinema_spending_pp + google_trend_ott + cpi_all + cpi_culture + covid_cases +
    trend + season)
summary(full_tslm)
```

```
Call:
```

```
tslm(formula = cinema_admissions ~ cinema_screening + cinema_spending +
    avg_cinema_spending_pp + google_trend_ott + cpi_all + cpi_culture +
    covid_cases + trend + season)

Residuals:
     Min       1Q   Median       3Q      Max
-1250170  -137734   -29132   143396   975312

Coefficients:
                         Estimate Std. Error t value Pr(>|t|)
(Intercept)             6.081e+06  3.522e+05  17.268  < 2e-16 ***
cinema_screening        7.470e-01  5.223e-01   1.430 0.154257
cinema_spending         1.401e-01  1.782e-03  78.603  < 2e-16 ***
avg_cinema_spending_pp -8.683e+05  5.599e+04 -15.509  < 2e-16 ***
google_trend_ott       -7.570e+02  2.771e+03  -0.273 0.785028
cpi_all                 1.557e+05  7.050e+04   2.208 0.028387 *
cpi_culture            -1.115e+04  4.608e+04  -0.242 0.809083
covid_cases             6.777e-02  5.481e-02   1.236 0.217764
trend                  -3.161e+03  1.052e+03  -3.005 0.003001 **
season2                 3.995e+05  1.163e+05   3.436 0.000721 ***
season3                 4.229e+05  1.195e+05   3.540 0.000501 ***
season4                 2.790e+05  1.215e+05   2.296 0.022731 *
season5                 3.148e+05  1.334e+05   2.360 0.019281 *
season6                 2.106e+05  1.470e+05   1.432 0.153695
season7                 2.040e+05  1.484e+05   1.374 0.170893
season8                 1.764e+05  1.444e+05   1.222 0.223219
season9                 3.531e+05  1.330e+05   2.654 0.008603 **
season10                3.104e+05  1.186e+05   2.617 0.009554 **
season11                6.078e+05  1.157e+05   5.255 3.85e-07 ***
season12                3.296e+05  1.119e+05   2.946 0.003612 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 281200 on 196 degrees of freedom
Multiple R-squared:  0.9954,    Adjusted R-squared:  0.995
F-statistic:  2257 on 19 and 196 DF,  p-value: < 2.2e-16
```

```r
checkresiduals(full_tslm)
```




```
    Breusch-Godfrey test for serial correlation of order up to 24

data:  Residuals from Linear regression model
LM test = 68.792, df = 24, p-value = 3.327e-06
```

```
model_name <- "TSLM - Full"
aic <- sprintf("%.3f", AIC(full_tslm))
dw <- sprintf("%.3f", as.numeric(dwtest(full_tslm)$statistic))
lj_box <- NA
r2 <- sprintf("%.3f", summary(full_tslm)$r.squared)
acc <- as.data.frame.array(forecast::accuracy(full_tslm))
mae <- format(acc$MAE, big.mark = ",", nsmall = 2, scientific = FALSE)
rmse <- format(acc$RMSE, big.mark = ",", nsmall = 2, scientific = FALSE)



model_performance[nrow(model_performance) + 1, ] <- c(model_name, aic, dw,
    lj_box, r2, mae, rmse)
```

Stepwise Regression

```
# Convert tslm object to lm object
full_lm <- lm(formula(full_tslm), data = as.data.frame(ott_cinema_monthly.ts))

# Stepwise Selection (Both Directions: Forward & Backward)
stepwise_model <- step(full_lm, direction = "both", trace = TRUE)
```

```
Start:  AIC=5439.29
```

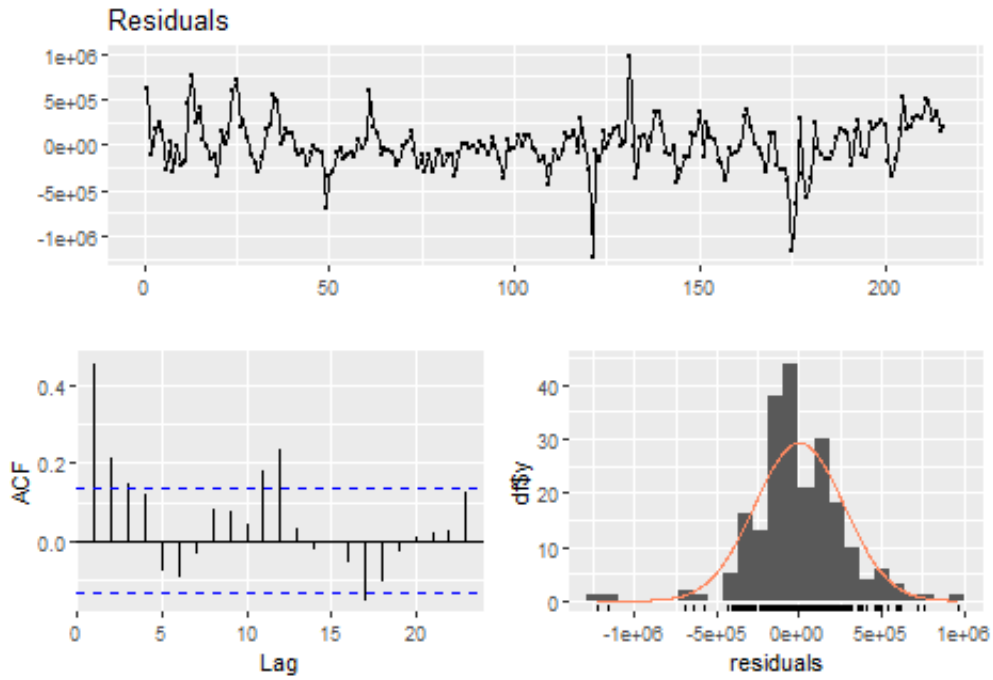
```
cinema_admissions ~ cinema_screening + cinema_spending + avg_cinema_spending_pp +
    google_trend_ott + cpi_all + cpi_culture + covid_cases +
    trend + season

                         Df  Sum of Sq        RSS    AIC
- cpi_culture             1 4.6294e+09 1.5506e+13 5437.4
- google_trend_ott        1 5.9009e+09 1.5508e+13 5437.4
- covid_cases             1 1.2092e+11 1.5623e+13 5439.0
<none>                                1.5502e+13 5439.3
- cinema_screening        1 1.6177e+11 1.5664e+13 5439.5
- cpi_all                 1 3.8567e+11 1.5888e+13 5442.6
- trend                   1 7.1430e+11 1.6216e+13 5447.0
- season                 11 3.5230e+12 1.9025e+13 5461.5
- avg_cinema_spending_pp  1 1.9024e+13 3.4525e+13 5610.2
- cinema_spending         1 4.8865e+14 5.0415e+14 6189.4

Step:  AIC=5437.35
cinema_admissions ~ cinema_screening + cinema_spending + avg_cinema_spending_pp +
    google_trend_ott + cpi_all + covid_cases + trend + season

                         Df  Sum of Sq        RSS    AIC
- google_trend_ott        1 7.0217e+09 1.5513e+13 5435.4
- covid_cases             1 1.2145e+11 1.5628e+13 5437.0
<none>                                1.5506e+13 5437.4
- cinema_screening        1 1.6011e+11 1.5667e+13 5437.6
+ cpi_culture             1 4.6294e+09 1.5502e+13 5439.3
- cpi_all                 1 3.8115e+11 1.5888e+13 5440.6
- trend                   1 7.1058e+11 1.6217e+13 5445.0
- season                 11 3.5906e+12 1.9097e+13 5460.3
- avg_cinema_spending_pp  1 1.9144e+13 3.4651e+13 5609.0
- cinema_spending         1 4.9224e+14 5.0775e+14 6188.9

Step:  AIC=5435.45
cinema_admissions ~ cinema_screening + cinema_spending + avg_cinema_spending_pp +
    cpi_all + covid_cases + trend + season

                         Df  Sum of Sq        RSS    AIC
- covid_cases             1 1.1562e+11 1.5629e+13 5435.1
<none>                                1.5513e+13 5435.4
+ google_trend_ott        1 7.0217e+09 1.5506e+13 5437.4
+ cpi_culture             1 5.7502e+09 1.5508e+13 5437.4
- cinema_screening        1 3.3315e+11 1.5847e+13 5438.0
- cpi_all                 1 3.7529e+11 1.5889e+13 5438.6
- season                 11 3.6136e+12 1.9127e+13 5458.7
- trend                   1 3.3740e+12 1.8888e+13 5476.0
- avg_cinema_spending_pp  1 1.9306e+13 3.4819e+13 5608.1
- cinema_spending         1 4.9279e+14 5.0830e+14 6187.2

Step:  AIC=5435.05
cinema_admissions ~ cinema_screening + cinema_spending + avg_cinema_spending_pp +
    cpi_all + trend + season

                         Df  Sum of Sq        RSS    AIC
<none>                                1.5629e+13 5435.1
```

```
+ covid_cases             1 1.1562e+11 1.5513e+13 5435.4
+ cpi_culture             1 5.5972e+09 1.5624e+13 5437.0
+ google_trend_ott        1 1.1910e+09 1.5628e+13 5437.0
- cinema_screening        1 3.5284e+11 1.5982e+13 5437.9
- cpi_all                 1 7.0313e+11 1.6332e+13 5442.6
- season                 11 3.6449e+12 1.9274e+13 5458.3
- trend                   1 3.2813e+12 1.8910e+13 5474.2
- avg_cinema_spending_pp  1 1.9229e+13 3.4858e+13 5606.3
- cinema_spending         1 5.2254e+14 5.3817e+14 6197.5
```

```r
# Display Summary of Selected Model
summary(stepwise_model)
```

```
Call:
lm(formula = cinema_admissions ~ cinema_screening + cinema_spending +
    avg_cinema_spending_pp + cpi_all + trend + season, data = as.data.frame(ott_cinema_monthly.ts))

Residuals:
     Min       1Q   Median       3Q      Max
-1227239  -135008   -25772   146570   969888

Coefficients:
                         Estimate Std. Error t value Pr(>|t|)
(Intercept)             6.119e+06  3.489e+05  17.539  < 2e-16 ***
cinema_screening        8.629e-01  4.071e-01   2.120 0.035281 *
cinema_spending         1.396e-01  1.711e-03  81.568  < 2e-16 ***
avg_cinema_spending_pp -8.683e+05  5.549e+04 -15.647  < 2e-16 ***
cpi_all                 1.879e+05  6.280e+04   2.992 0.003121 **
trend                  -3.336e+03  5.162e+02  -6.464 7.71e-10 ***
season2                 3.720e+05  1.093e+05   3.402 0.000808 ***
season3                 3.953e+05  1.146e+05   3.448 0.000689 ***
season4                 2.518e+05  1.161e+05   2.169 0.031258 *
season5                 2.868e+05  1.234e+05   2.324 0.021133 *
season6                 1.702e+05  1.336e+05   1.274 0.204004
season7                 1.695e+05  1.306e+05   1.298 0.195835
season8                 1.319e+05  1.248e+05   1.057 0.291772
season9                 3.430e+05  1.229e+05   2.790 0.005777 **
season10                2.784e+05  1.121e+05   2.483 0.013871 *
season11                5.909e+05  1.126e+05   5.248 3.93e-07 ***
season12                2.984e+05  9.822e+04   3.038 0.002698 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 280200 on 199 degrees of freedom
Multiple R-squared:  0.9954,    Adjusted R-squared:  0.995
F-statistic:  2699 on 16 and 199 DF,  p-value: < 2.2e-16
```

```r
checkresiduals(stepwise_model)
```



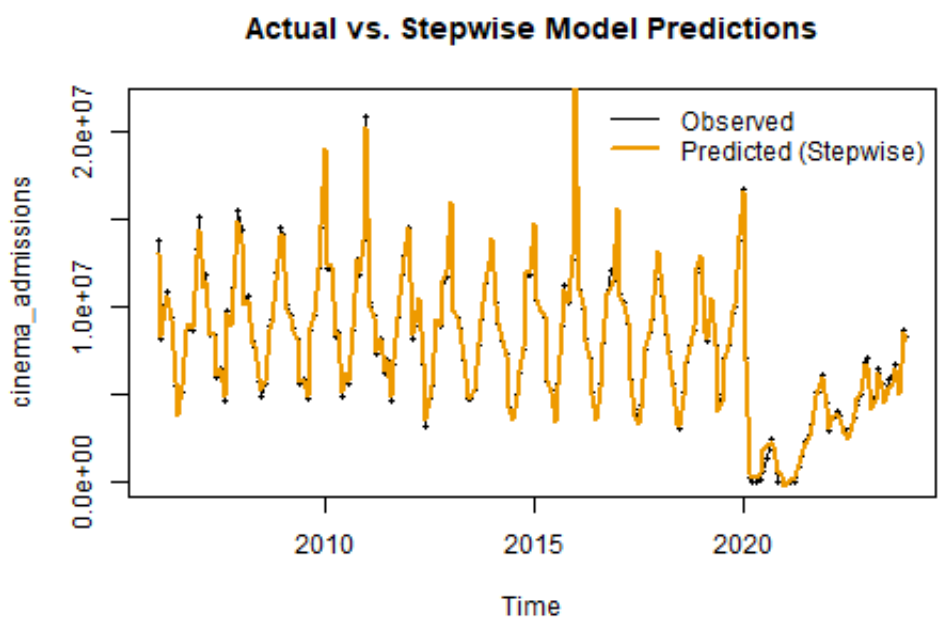

```
    Breusch-Godfrey test for serial correlation of order up to 20

data:  Residuals
LM test = 68.12, df = 20, p-value = 3.682e-07
```

```r
dwtest(stepwise_model)  #indicates a problem of positive autocorrelation
```

```
    Durbin-Watson test

data:  stepwise_model
DW = 1.0635, p-value = 2.92e-12
alternative hypothesis: true autocorrelation is greater than 0
```

```r
model_name <- "TSLM - Stepwise"
aic <- sprintf("%.3f", AIC(stepwise_model))
dw <- sprintf("%.3f", as.numeric(dwtest(stepwise_model)$statistic))
lj_box <- NA
r2 <- sprintf("%.3f", summary(stepwise_model)$r.squared)
acc <- as.data.frame.array(forecast::accuracy(stepwise_model))
mae <- format(acc$MAE, big.mark = ",", nsmall = 2, scientific = FALSE)
rmse <- format(acc$RMSE, big.mark = ",", nsmall = 2, scientific = FALSE)

model_performance[nrow(model_performance) + 1, ] <- c(model_name, aic, dw,
    lj_box, r2, mae, rmse)
```

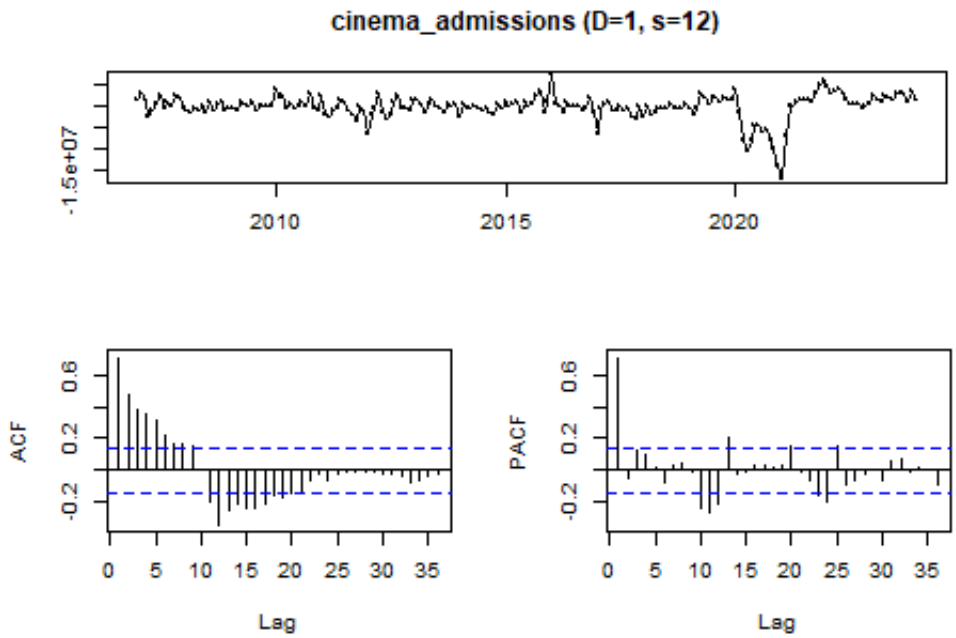
```r
# Extract fitted values from stepwise model
fitted_stepwise <- fitted(stepwise_model)

# Convert fitted values into a time series object
fitted_stepwise.ts <- ts(fitted_stepwise, start = start(cinema_admissions),
    frequency = frequency(cinema_admissions))


plot(cinema_admissions, type = "o", pch = 16, cex = 0.6, main = "Actual vs. Stepwise Model Predictions")

# Add Stepwise Regression Predictions
lines(fitted_stepwise.ts, col = "orange2", lwd = 2, lty = 1)

# Add Legend
legend("topright", legend = c("Observed", "Predicted (Stepwise)"), col = c("black",
    "orange2"), lwd = c(1, 2), lty = c(1, 1), bty = "n")
```



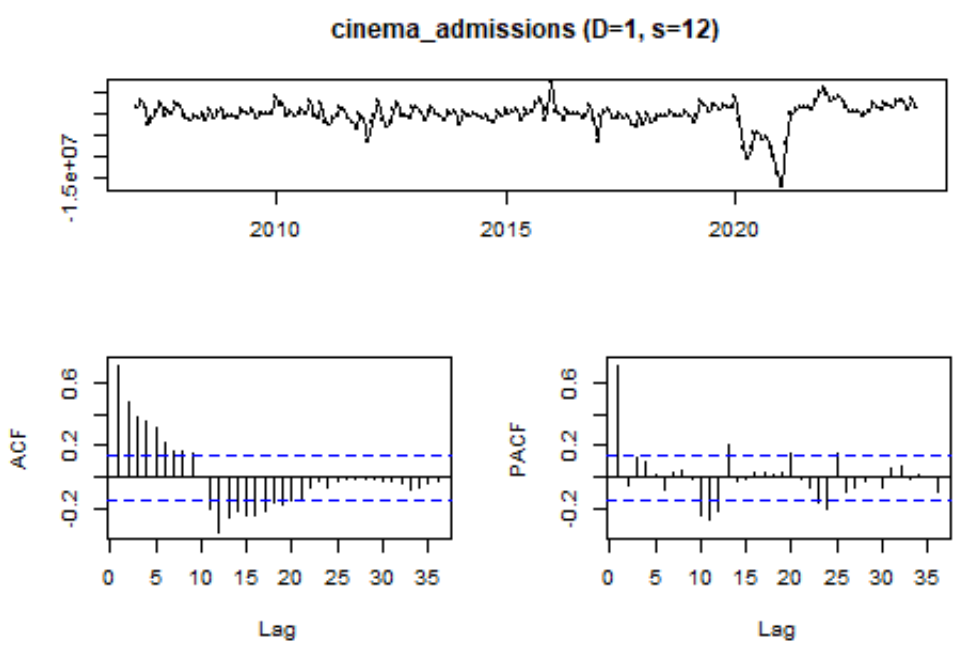
ARIMA Models

General indication (for differenced data): - If the ACF is exponentially decaying or sinusoidal and there is a significant spike at lag p in PACF and nothing else, it may be an ARIMA(p,d,0). - If the PACF is exponentially decaying or sinusoidal and there is a significant spike at lag q in ACF and nothing else, it may be an ARIMA(0,d,q).

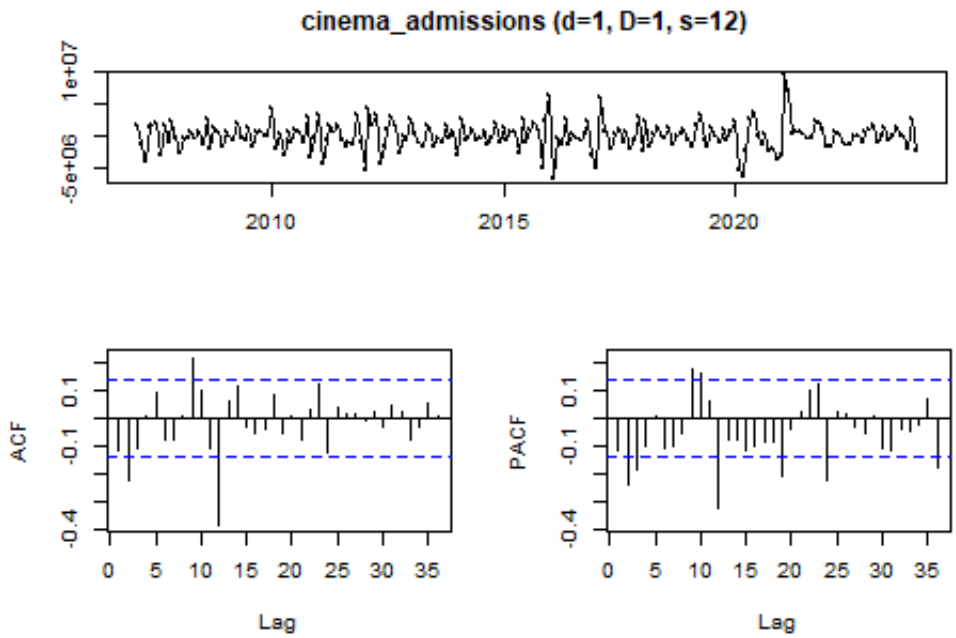
```r
diff1 <- diff(cinema_admissions)
tsdisplay(diff1, main = "cinema_admissions (d=1)")
```




```
diff2 <- diff(cinema_admissions, lag = 12)
tsdisplay(diff2, main = "cinema_admissions (D=1, s=12)")
```




```
diff3 <- diff(diff1, lag = 12)
tsdisplay(diff3, main = "cinema_admissions (d=1, D=1, s=12)")
```

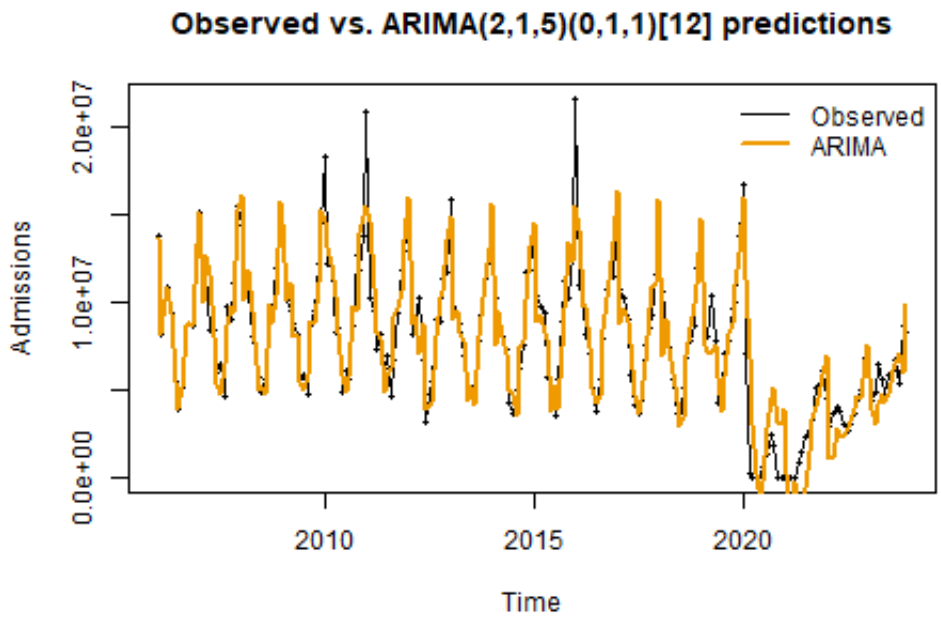


ARIMA

```
# first Arima model
arima1.cinema_admissions <- Arima(cinema_admissions, order = c(2, 1, 5),
    seasonal = c(0, 1, 1))
summary(arima1.cinema_admissions)
```

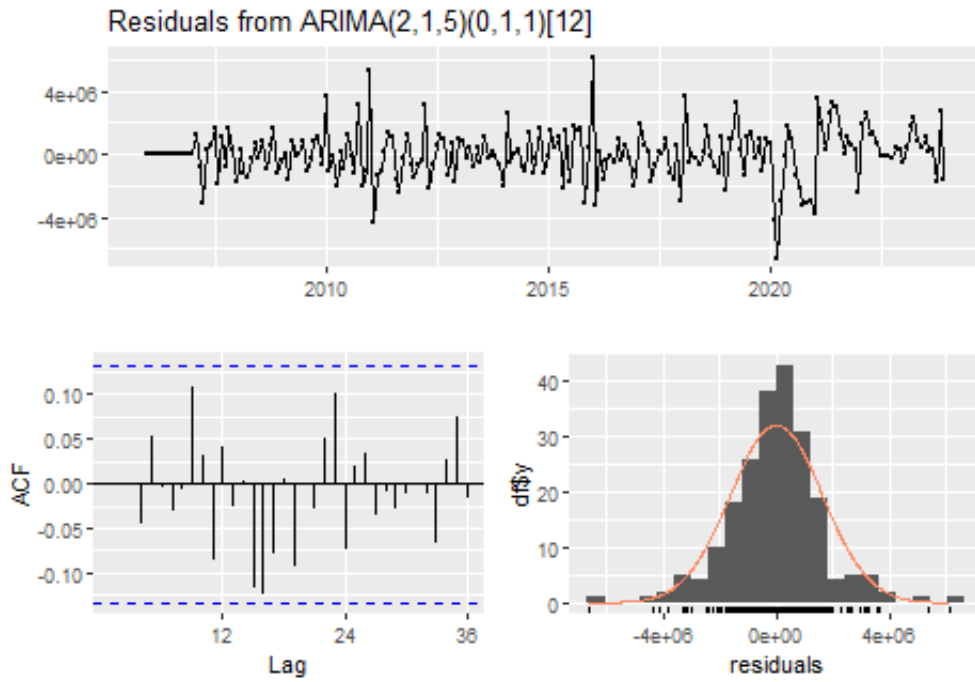
```
Series: cinema_admissions
ARIMA(2,1,5)(0,1,1)[12]

Coefficients:
         ar1      ar2      ma1     ma2      ma3      ma4      ma5     sma1
      0.9343  -0.8117  -1.2506  0.9493  -0.1235  -0.0996  -0.1444  -0.6638
s.e.  0.0842   0.0763   0.1024  0.1448   0.1504   0.1346   0.0837   0.0781

sigma^2 = 2.913e+12:  log likelihood = -3202.4
AIC=6422.8   AICc=6423.73   BIC=6452.62

Training set error measures:
                    ME    RMSE     MAE MPE MAPE      MASE        ACF1
Training set -7762.615 1621662 1171317 NaN  Inf 0.6590269 0.001435097
```

```
fit.arima1.cinema_admissions <- fitted(arima1.cinema_admissions)

plot(cinema_admissions, type = "o", pch = 16, cex = 0.6, ylab = "Admissions",
    main = "Observed vs. ARIMA(2,1,5)(0,1,1)[12] predictions")
lines(fit.arima1.cinema_admissions, col = "orange2", lwd = 2)
legend("topright", legend = c("Observed", "ARIMA"), col = c("black", "orange2"),
    lwd = c(1, 2), lty = c(1, 1), bty = "n")
```




```
res.arima1.cinema_admissions <- residuals(arima1.cinema_admissions)
tsdisplay(res.arima1.cinema_admissions)
```

```
model_name <- "ARIMA(2,1,5)(0,1,1)[12]"
aic <- sprintf("%.3f", AIC(arima1.cinema_admissions))
dw <- NA
lj_box <- sprintf("%.3f", checkresiduals(arima1.cinema_admissions)$p.value)
```




```
    Ljung-Box test

data:  Residuals from ARIMA(2,1,5)(0,1,1)[12]
Q* = 21.496, df = 16, p-value = 0.1602

Model df: 8.   Total lags used: 24
```

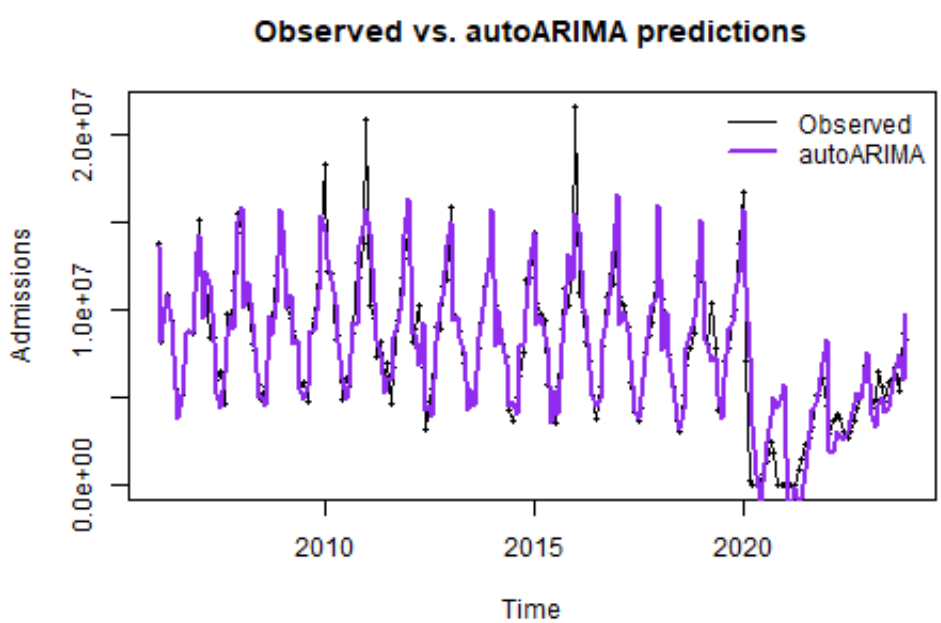
```
r2 <- sprintf("%.3f", R2_Score(fit.arima1.cinema_admissions, cinema_admissions))
acc <- as.data.frame.array(forecast::accuracy(arima1.cinema_admissions))
mae <- format(acc$MAE, big.mark = ",", nsmall = 2, scientific = FALSE)
rmse <- format(acc$RMSE, big.mark = ",", nsmall = 2, scientific = FALSE)

model_performance[nrow(model_performance) + 1, ] <- c(model_name, aic, dw,
    lj_box, r2, mae, rmse)
```

```
arima.a.cinema_admissions <- auto.arima(cinema_admissions)
summary(arima.a.cinema_admissions)
```

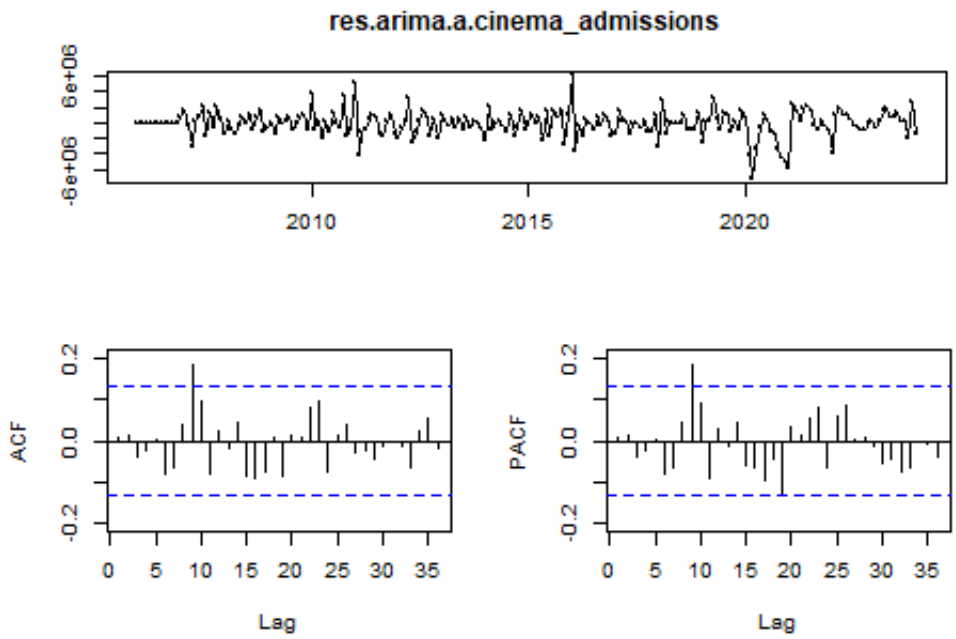
```
Series: cinema_admissions
ARIMA(1,0,2)(0,1,1)[12]

Coefficients:
         ar1      ma1      ma2     sma1
      0.8946  -0.2130  -0.1664  -0.6699
s.e.  0.0531   0.0939   0.0920   0.0668

sigma^2 = 2.931e+12:  log likelihood = -3219.36
AIC=6448.71   AICc=6449.01   BIC=6465.3
```

```
Training set error measures:
                    ME    RMSE     MAE MPE MAPE      MASE        ACF1
Training set -98286.64 1647445 1158030 NaN  Inf 0.6515513 0.007339816
```

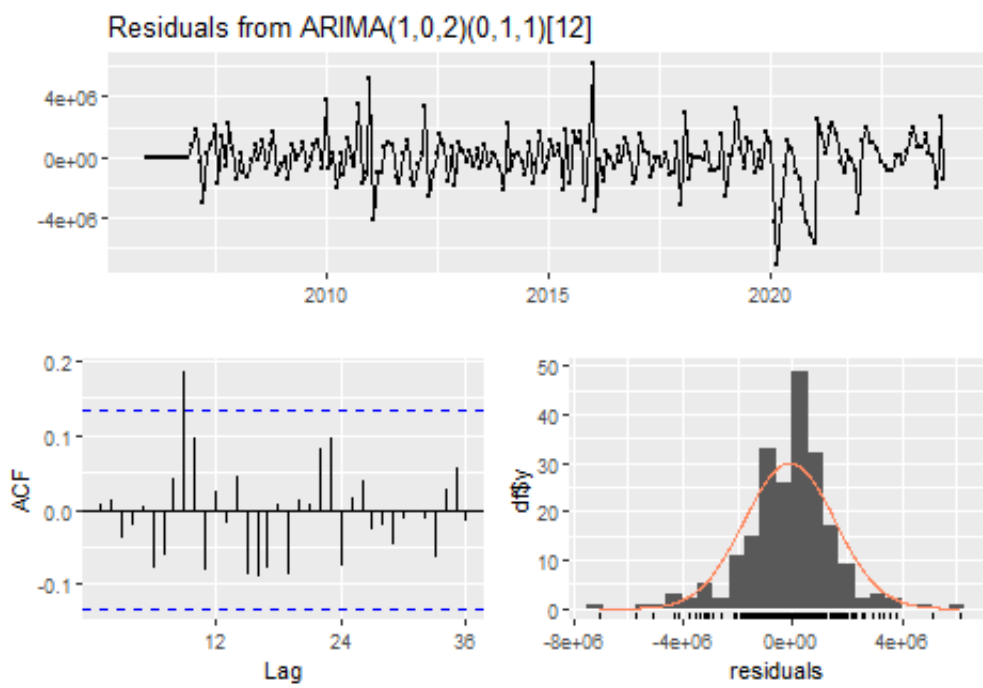
```r
fit.arima.a.cinema_admissions <- fitted(arima.a.cinema_admissions)

plot(cinema_admissions, type = "o", pch = 16, cex = 0.6, ylab = "Admissions",
    main = "Observed vs. autoARIMA predictions")
lines(fit.arima.a.cinema_admissions, col = "purple2", lwd = 2)
legend("topright", legend = c("Observed", "autoARIMA"), col = c("black",
    "purple2"), lwd = c(1, 2), lty = c(1, 1), bty = "n")
```




```r
res.arima.a.cinema_admissions <- residuals(arima.a.cinema_admissions)
tsdisplay(res.arima.a.cinema_admissions)
```




```
model_name <- "auto.ARIMA(1,0,2)(0,1,1)[12]"
aic <- sprintf("%.3f", AIC(arima.a.cinema_admissions))
dw <- NA
lj_box <- sprintf("%.3f", checkresiduals(arima.a.cinema_admissions)$p.value)
```




```
Ljung-Box test
```

```
data:  Residuals from ARIMA(1,0,2)(0,1,1)[12]
Q* = 28.071, df = 20, p-value = 0.1077

Model df: 4.   Total lags used: 24
```

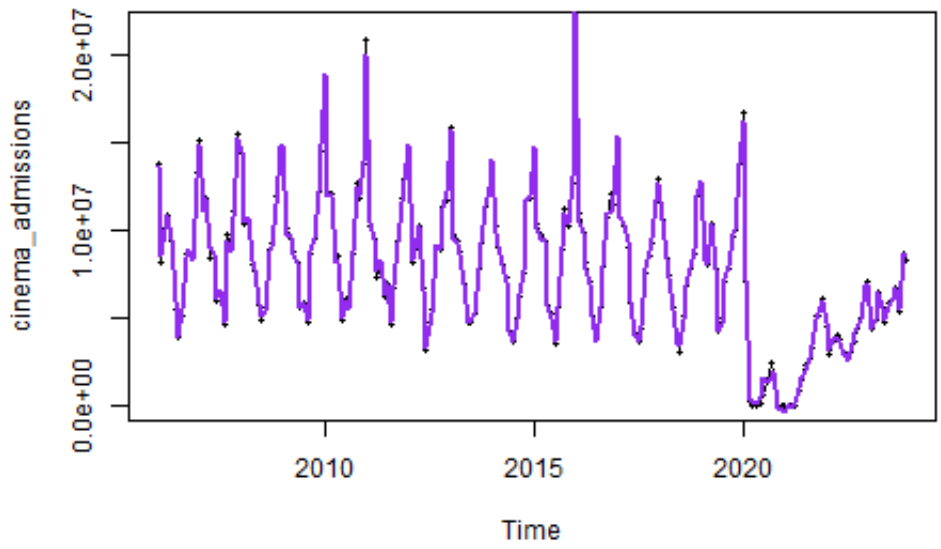
```r
r2 <- sprintf("%.3f", R2_Score(fit.arima.a.cinema_admissions, cinema_admissions))
acc <- as.data.frame.array(forecast::accuracy(arima.a.cinema_admissions))
mae <- format(acc$MAE, big.mark = ",", nsmall = 2, scientific = FALSE)
rmse <- format(acc$RMSE, big.mark = ",", nsmall = 2, scientific = FALSE)

model_performance[nrow(model_performance) + 1, ] <- c(model_name, aic, dw,
    lj_box, r2, mae, rmse)
```

ARIMAx

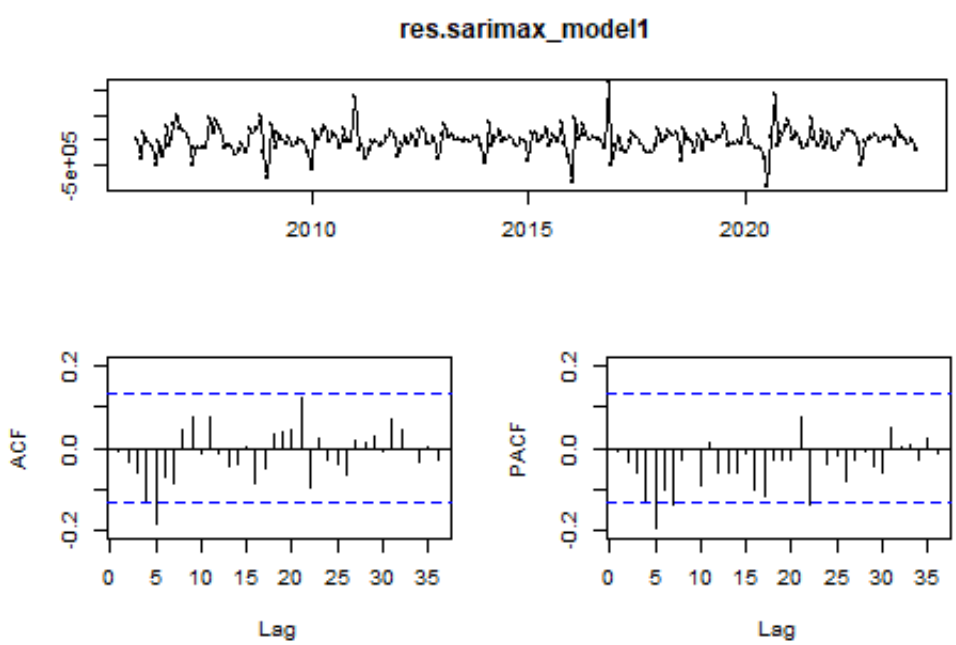
```r
xreg1 <- cbind(cinema_screening, cinema_spending, avg_cinema_spending_pp,
    cpi_all)

sarimax_model1 <- auto.arima(cinema_admissions, xreg = xreg1)
summary(sarimax_model1)
```

```
Series: cinema_admissions
Regression with ARIMA(3,1,0)(2,0,1)[12] errors

Coefficients:
          ar1      ar2      ar3    sar1    sar2     sma1  cinema_screening
      -0.4251  -0.3035  -0.1890  0.7920  0.0096  -0.4090            2.2637
s.e.   0.0674   0.0712   0.0685  0.2474  0.1549   0.2413            0.6136
      cinema_spending  avg_cinema_spending_pp   cpi_all
               0.1358              -837865.17  43073.56
s.e.           0.0016                45654.96  51621.39

sigma^2 = 6.334e+10:  log likelihood = -2976.33
AIC=5974.66   AICc=5975.96   BIC=6011.73

Training set error measures:
                    ME     RMSE      MAE MPE MAPE       MASE         ACF1
Training set -3309.766 245186.5 170544.1 NaN  Inf 0.09595449 -0.008151537
```

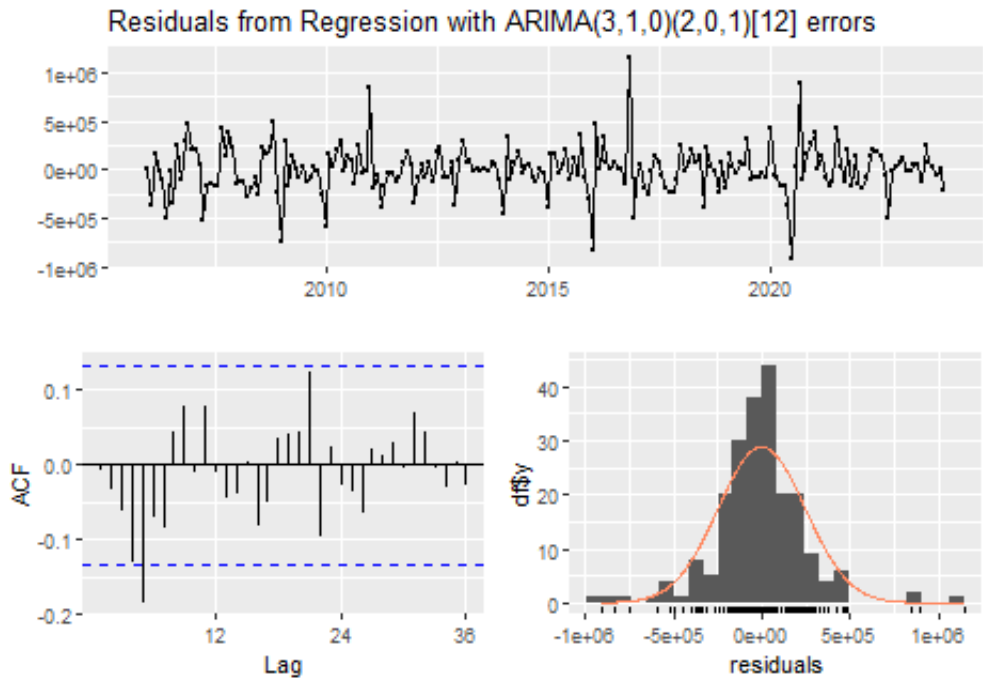
```r
fit.sarimax1 <- fitted(sarimax_model1)

plot(cinema_admissions, type = "o", pch = 16, cex = 0.6)
lines(fit.sarimax1, col = "purple2", lwd = 2)
```




```
res.sarimax_model1 <- residuals(sarimax_model1)
tsdisplay(res.sarimax_model1)
```




```
model_name <- "auto.ARIMAx(3,1,0)(2,0,1)[12]"
aic <- sprintf("%.3f", AIC(sarimax_model1))
dw <- NA
lj_box <- sprintf("%.3f", checkresiduals(sarimax_model1)$p.value)
```




```
    Ljung-Box test

data:  Residuals from Regression with ARIMA(3,1,0)(2,0,1)[12] errors
Q* = 29.078, df = 18, p-value = 0.04744

Model df: 6.   Total lags used: 24
```

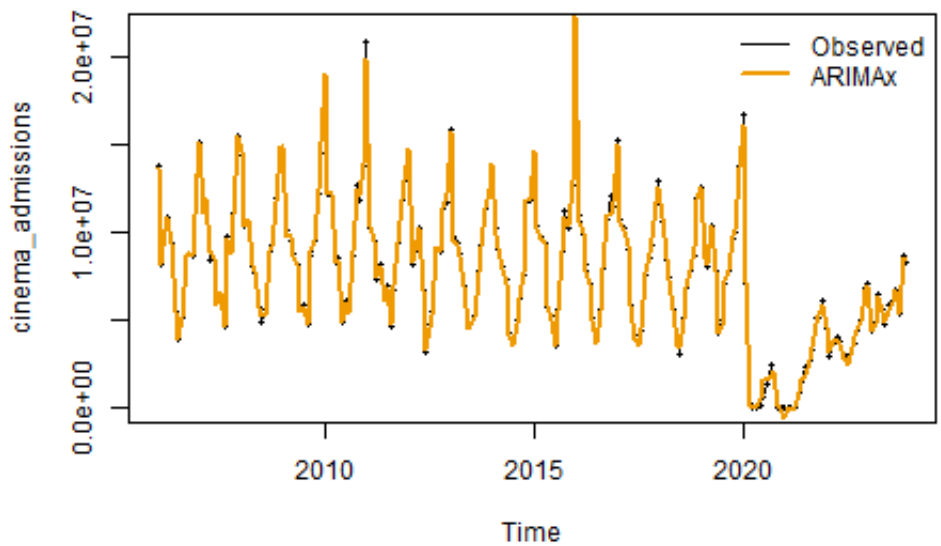
```
r2 <- sprintf("%.3f", R2_Score(fit.sarimax1, cinema_admissions))
acc <- as.data.frame.array(forecast::accuracy(sarimax_model1))
mae <- format(acc$MAE, big.mark = ",", nsmall = 2, scientific = FALSE)
rmse <- format(acc$RMSE, big.mark = ",", nsmall = 2, scientific = FALSE)

model_performance[nrow(model_performance) + 1, ] <- c(model_name, aic, dw,
    lj_box, r2, mae, rmse)
```

```
sarimax_model2 <- Arima(cinema_admissions, order = c(1, 1, 2), seasonal = c(0,
    1, 1), xreg = xreg1)
summary(sarimax_model2)
```

```
Series: cinema_admissions
Regression with ARIMA(1,1,2)(0,1,1)[12] errors

Coefficients:
         ar1      ma1     ma2     sma1  cinema_screening  cinema_spending
      0.3925  -0.9626  0.0112  -0.5549            2.8627           0.1349
s.e.  0.1959   0.2122  0.1925   0.0710            0.5248           0.0017
      avg_cinema_spending_pp   cpi_all
                   -826145.4  67405.67
s.e.                 45770.3  50562.68
```

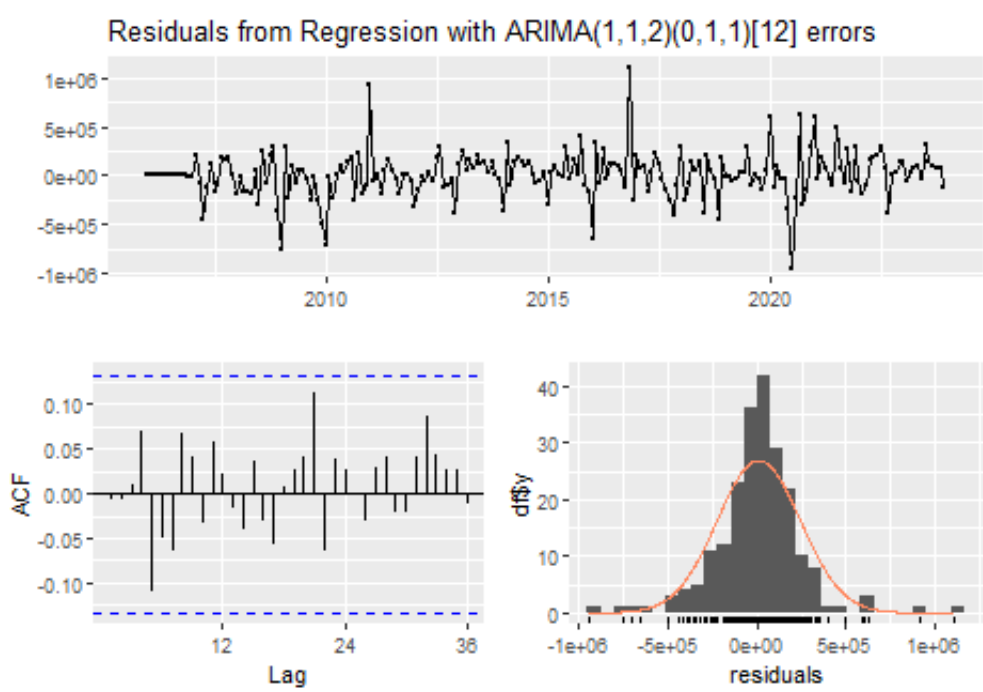
```
sigma^2 = 6.065e+10:  log likelihood = -2807.4
AIC=5632.8   AICc=5633.74   BIC=5662.62

Training set error measures:
                   ME   RMSE      MAE MPE MAPE       MASE         ACF1
Training set 9637.047 233991 159146.6 NaN  Inf 0.08954183 -0.006528331
```

```r
fit.sarimax2 <- fitted(sarimax_model2)

plot(cinema_admissions, type = "o", pch = 16, cex = 0.6)
lines(fit.sarimax2, col = "orange2", lwd = 2)
legend("topright", legend = c("Observed", "ARIMAx"), col = c("black", "orange2"),
    lwd = c(1, 2), lty = c(1, 1), bty = "n")
```




```r
res.sarimax_model2 <- residuals(sarimax_model2)
tsdisplay(res.sarimax_model2)
```

```
model_name <- "ARIMAx(1,1,2)(0,1,1)[12]"
aic <- sprintf("%.3f", AIC(sarimax_model2))
dw <- NA
lj_box <- sprintf("%.3f", checkresiduals(sarimax_model2)$p.value)
```




```
Ljung-Box test
```

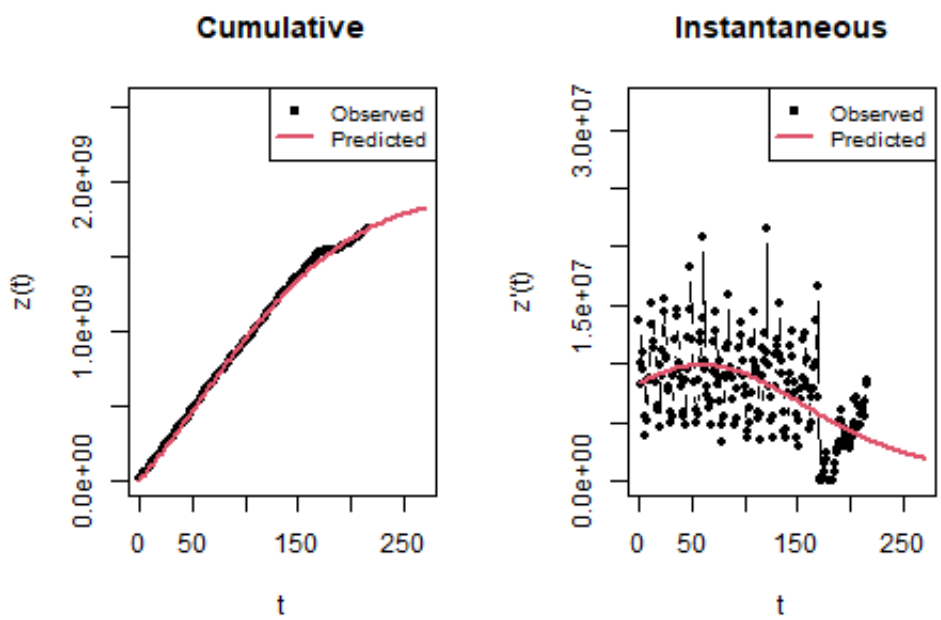
```
data:  Residuals from Regression with ARIMA(1,1,2)(0,1,1)[12] errors
Q* = 14.895, df = 20, p-value = 0.7824

Model df: 4.   Total lags used: 24
```

```
r2 <- sprintf("%.3f", R2_Score(fit.sarimax2, cinema_admissions))
acc <- as.data.frame.array(forecast::accuracy(sarimax_model2))
mae <- format(acc$MAE, big.mark = ",", nsmall = 2, scientific = FALSE)
rmse <- format(acc$RMSE, big.mark = ",", nsmall = 2, scientific = FALSE)

model_performance[nrow(model_performance) + 1, ] <- c(model_name, aic, dw,
    lj_box, r2, mae, rmse)
```

Diffusion Models

Bass Model (BM)

```
# Fit the Bass Model
BM.cinema_admissions <- BM(cinema_admissions)
```

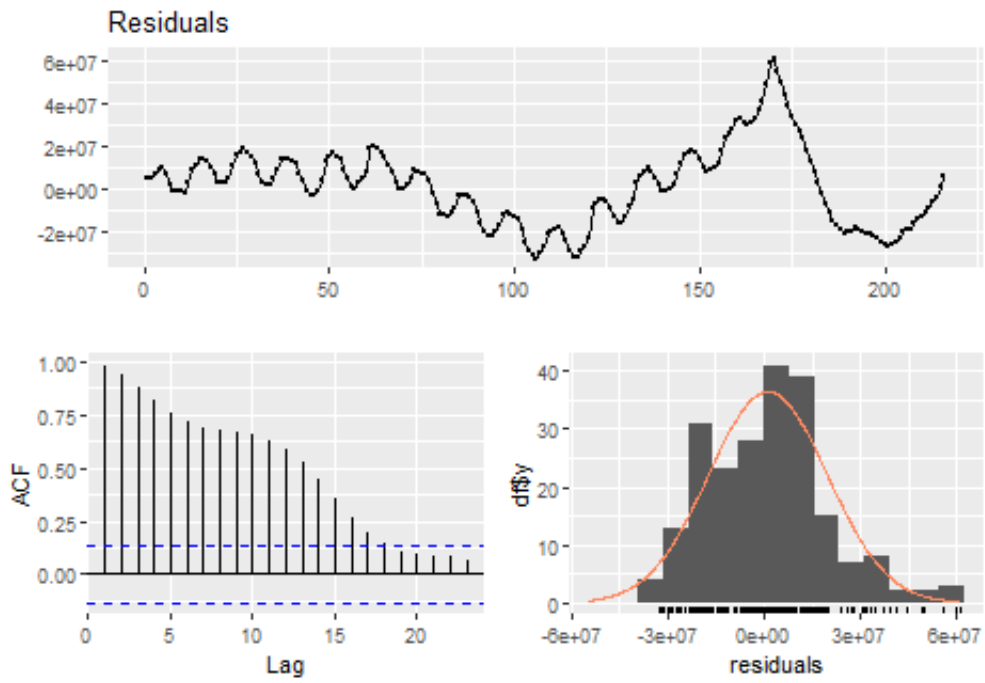



```
summary(BM.cinema_admissions)
```

```
Call: ( Standard Bass Model )

  BM(series = cinema_admissions)

Residuals:
      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
```

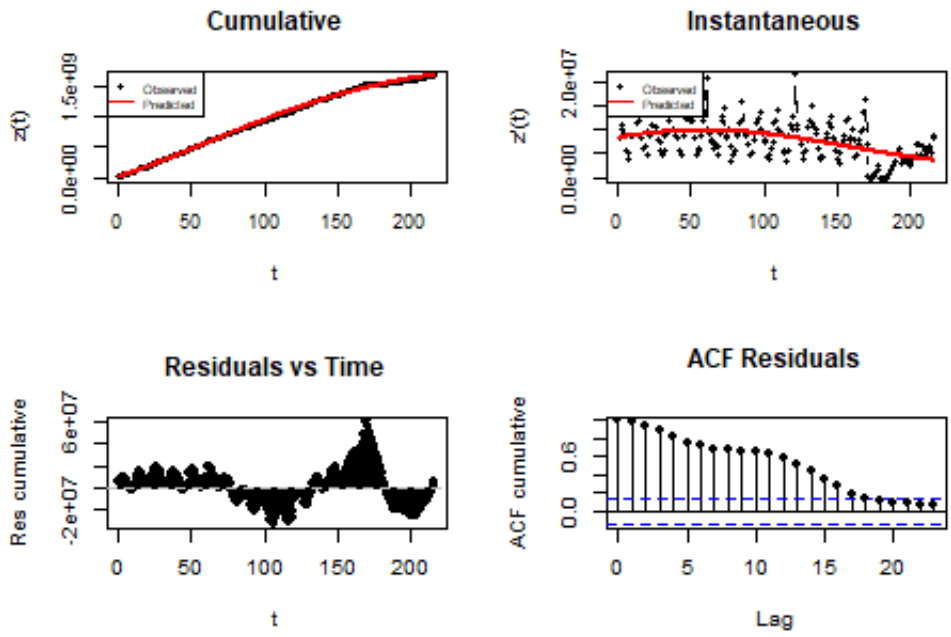
```
-32795351 -13185860   2400549   1195905  11867557  61205017

Coefficients:
       Estimate    Std.Error        Lower        Upper   p-value
m 1.963262e+09 1.836356e+07 1.927270e+09 1.999254e+09 4.77e-187 ***
p 4.245253e-03 3.418230e-05 4.178257e-03 4.312249e-03 1.08e-200 ***
q 9.821809e-03 3.598327e-04 9.116550e-03 1.052707e-02  1.76e-71 ***
---
 Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

 Residual standard error  18656992  on  213  degrees of freedom
 Multiple R-squared:   0.999705  Residual sum of squares:  7.414175e+16
```

```r
checkresiduals(BM.cinema_admissions)
```




```
    Ljung-Box test

data:  Residuals
Q* = 1378.5, df = 10, p-value < 2.2e-16

Model df: 0.   Total lags used: 10
```

```r
plot.Dimora(BM.cinema_admissions)
```



Prediction

```r
# Generate time index for observed data (2006 to 2023, monthly)
observed_months <- seq(as.Date("2006-01-01"), as.Date("2023-12-01"), by = "month")
# Generate time index for predictions (2006 to 2033, monthly)
#predicted_months <- seq(as.Date("2006-01-01"), as.Date("2033-12-01"), by = "month")

# Set prediction horizon
forecast_horizon <- 12*5

# Start predictions from the first month after observed data ends
predicted_months <- seq(from = max(observed_months) + 31,  # Start from next month
                        by = "month",
                        length.out = forecast_horizon)
```

```r
# Predict into the future
pred_BM.cinema_admissions <- predict(BM.cinema_admissions, newx = 1:length(observed_months))
inst_pred_BM.cinema_admissions <- make.instantaneous(pred_BM.cinema_admissions)
```

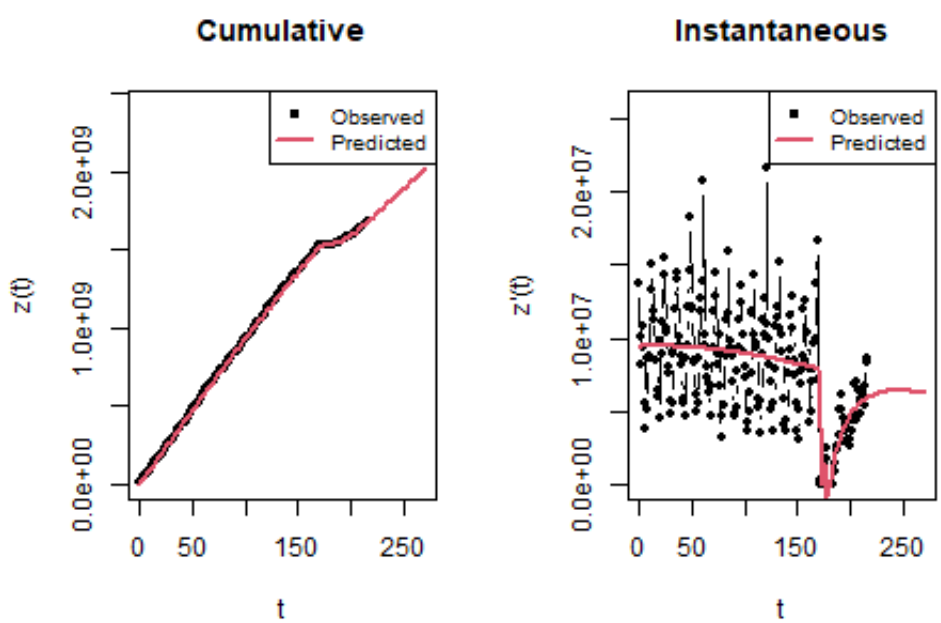
```r
model_name <- "Bass Model"
aic <- NA
dw <- NA
lj_box <- "< 2.2e-16"
r2 <- sprintf("%.3f", as.numeric(BM.cinema_admissions$Rsquared))
# acc <- as.data.frame.array(accuracy(sarimax_model2))
mae <- format(mean(abs(cinema_admissions - inst_pred_BM.cinema_admissions)),
    big.mark = ",", nsmall = 2, scientific = FALSE)
rmse <- format(sqrt(mean((cinema_admissions - inst_pred_BM.cinema_admissions)^2)),
```

```r
    big.mark = ",", nsmall = 2, scientific = FALSE)

model_performance[nrow(model_performance) + 1, ] <- c(model_name, aic, dw,
    lj_box, r2, mae, rmse)
```

```r
# Predict into the future
forecast_BM.cinema_admissions <- predict(BM.cinema_admissions, newx = 1:length(c(observed_months,
    predicted_months)))
inst_forecast_BM.cinema_admissions <- make.instantaneous(forecast_BM.cinema_admissions)
```

GBM

```r
# GBM exponential shock
GBM.e2 <- GBM(cinema_admissions, shock = "exp", nshock = 2, prelimestimates = c(1963349000,
    0.004244444, 0.009822524, 170, -0.1, -1, 178, -0.7, -0.5))
```

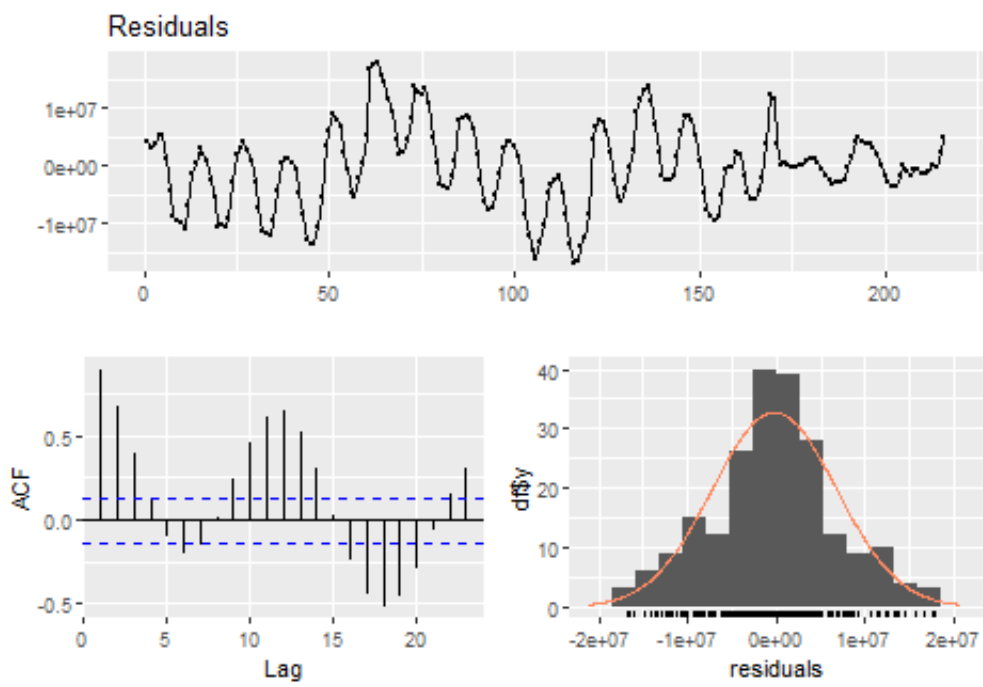



```r
# m,p,q, a(starting point), b(memory of the shock), c(intensity)
summary(GBM.e2)
```

```
Call: ( Generalized Bass model with 2  Exponential  shock )

  GBM(series = cinema_admissions, shock = "exp", nshock = 2, prelimestimates = c(1963349000,
    0.004244444, 0.009822524, 170, -0.1, -1, 178, -0.7, -0.5))

Residuals:
      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
-16755744  -4440017   -114140   -258674   3948145  17723321
```

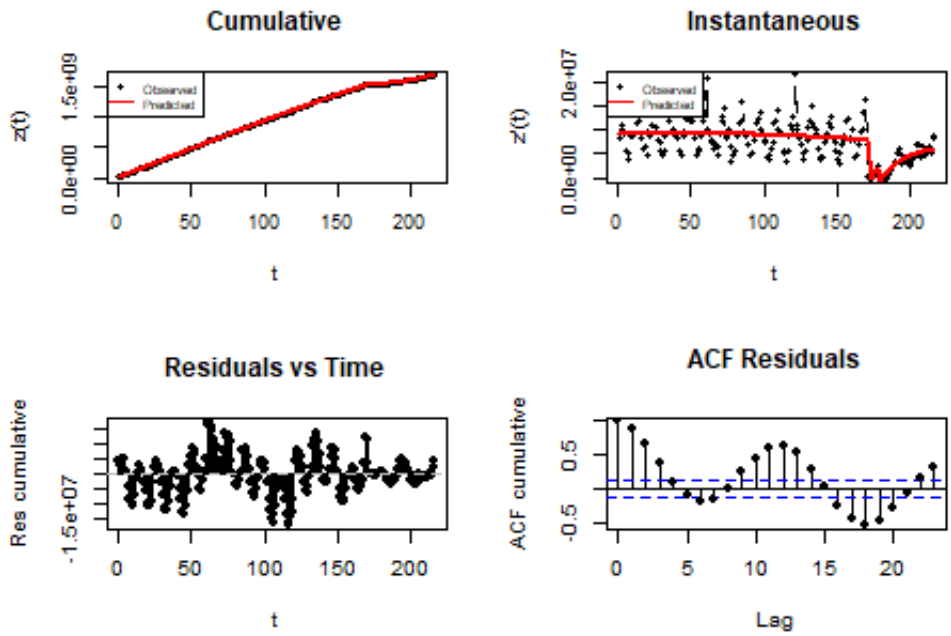
```
Coefficients:
           Estimate    Std.Error         Lower         Upper   p-value
m     3.502525e+09 2.169223e+08  3.077365e+09  3.927685e+09  1.70e-38 ***
p     2.689675e-03 1.524214e-04  2.390934e-03  2.988415e-03  3.88e-43 ***
q     2.964453e-03 3.723996e-04  2.234563e-03  3.694343e-03  1.12e-13 ***
a1    1.715302e+02 7.509239e-01  1.700584e+02  1.730019e+02 1.02e-250 ***
b1   -7.729553e-02 8.319385e-02 -2.403525e-01  8.576142e-02  3.54e-01
c1   -1.108408e+00 3.035016e-01 -1.703260e+00 -5.135557e-01  3.30e-04 ***
a2    1.778559e+02 2.047755e+00  1.738424e+02  1.818695e+02 8.05e-165 ***
b2   -2.586395e-02 1.276446e-02 -5.088183e-02 -8.460707e-04  4.40e-02   *
c2   -4.554099e-01 4.649267e-01 -1.366649e+00  4.558296e-01  3.28e-01
---
 Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

 Residual standard error  7121145  on  207  degrees of freedom
 Multiple R-squared:   0.999814  Residual sum of squares:  1.049712e+16
```

```r
checkresiduals(GBM.e2)
```




```
    Ljung-Box test

data:  Residuals
Q* = 391.2, df = 10, p-value < 2.2e-16

Model df: 0.   Total lags used: 10
```

```r
plot.Dimora(GBM.e2)
```




```r
# Predict into the future
pred_GBM.e2 <- predict(GBM.e2, newx = 1:length(observed_months))
inst_pred_GBM.e2 <- make.instantaneous(pred_GBM.e2)
```

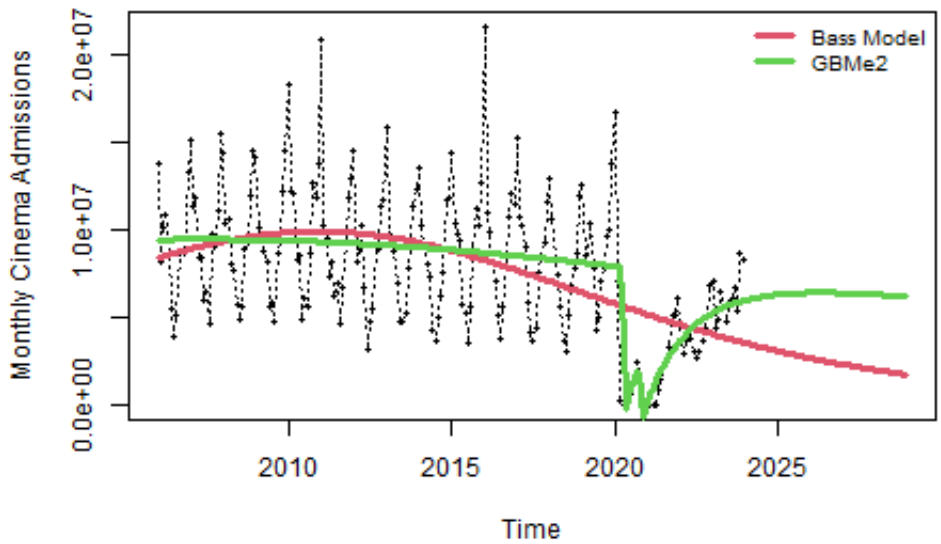
```r
model_name <- "GBMe2"
aic <- NA
dw <- NA
lj_box <- "< 2.2e-16"
r2 <- sprintf("%.3f", GBM.e2$Rsquared)
# acc <- as.data.frame.array(accuracy(sarimax_model2))
mae <- format(mean(abs(cinema_admissions - inst_pred_GBM.e2)), big.mark = ",",
    nsmall = 2, scientific = FALSE)
rmse <- format(sqrt(mean((cinema_admissions - inst_pred_GBM.e2)^2)), big.mark = ",",
    nsmall = 2, scientific = FALSE)

model_performance[nrow(model_performance) + 1, ] <- c(model_name, aic, dw,
    lj_box, r2, mae, rmse)
```

Prediction

```r
# Predict into the future
forecast_GBM.cinema_admissions <- predict(GBM.e2, newx = 1:length(c(observed_months,
    predicted_months)))
inst_forecast_GBM.cinema_admissions <- make.instantaneous(forecast_GBM.cinema_admissions)
```

```r
# Plot observed monthly cinema_admissions
plot(observed_months, cinema_admissions, type = "o", pch = 16, lty = 3, cex = 0.6,
    xlab = "Time", ylab = "Monthly Cinema Admissions", xlim = range(c(observed_months,
        predicted_months)), ylim = c(0, max(cinema_admissions, inst_forecast_GBM.cinema_admissions)))
```

```r
# Overlay predicted instantaneous cinema_admissions
lines(c(observed_months, predicted_months), inst_forecast_BM.cinema_admissions,
    lwd = 3, col = 2)
lines(c(observed_months, predicted_months), inst_forecast_GBM.cinema_admissions,
    lwd = 3, col = 3)

legend("topright", c("Bass Model", "GBMe2"), col = c(2, 3), lty = c(1, 1),
    lwd = c(3, 3), merge = TRUE, cex = 0.8, bty = "n")
```



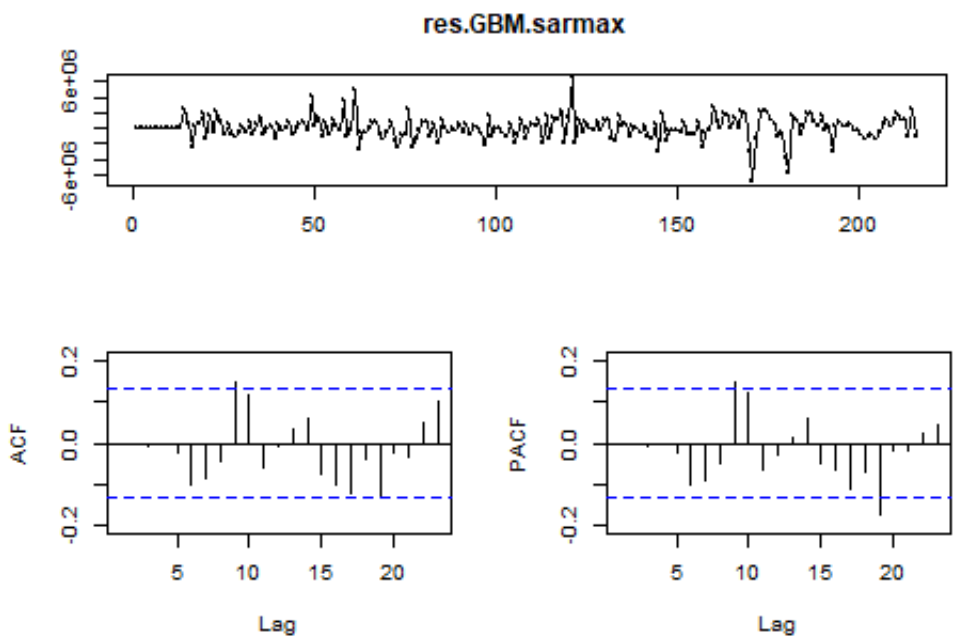
GBM with SARIMAx refinement

```r
# SARMAX Refinement
fit_GBM.e2 <- fitted(GBM.e2)
# inst_fit_GBM.cinema_admissions<-
# make.instantaneous(fit_GBM.cinema_admissions)

GBM.sarmax <- Arima(cumsum(cinema_admissions), order = c(2, 1, 0), seasonal = list(order = c(0,
    1, 2), period = 12), xreg = fit_GBM.e2)
summary(GBM.sarmax)
```

```
Series: cumsum(cinema_admissions)
Regression with ARIMA(2,1,0)(0,1,2)[12] errors

Coefficients:
         ar1      ar2     sma1     sma2    xreg
      0.4599  -0.1670  -0.5828  -0.1125  0.9972
s.e.  0.0705   0.0722   0.0775   0.0808  0.0792
```

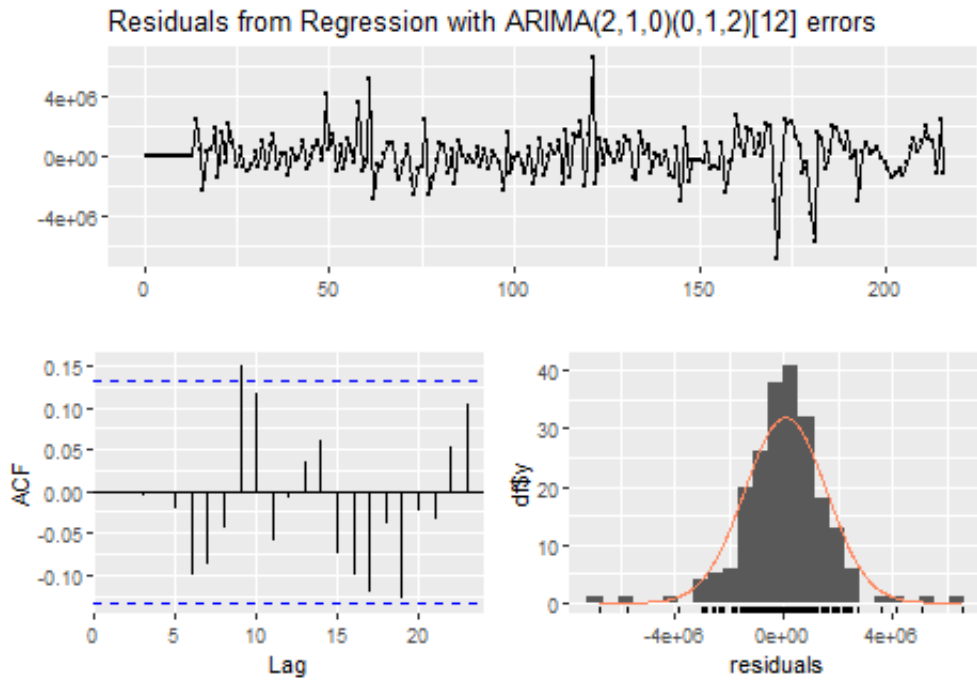
```
sigma^2 = 2.424e+12:  log likelihood = -3183.6
AIC=6379.21   AICc=6379.63   BIC=6399.09

Training set error measures:
                    ME    RMSE     MAE        MPE      MAPE      MASE
Training set 47332.79 1490790 1061866 0.01764457 0.1525263 0.1363268
                     ACF1
Training set -0.001565269
```

```r
res.GBM.sarmax <- residuals(GBM.sarmax)
tsdisplay(res.GBM.sarmax)
```




```r
fit.GBM.sarmax <- fitted(GBM.sarmax)
inst_fit.GBM.sarmax <- make.instantaneous(fit.GBM.sarmax)
```

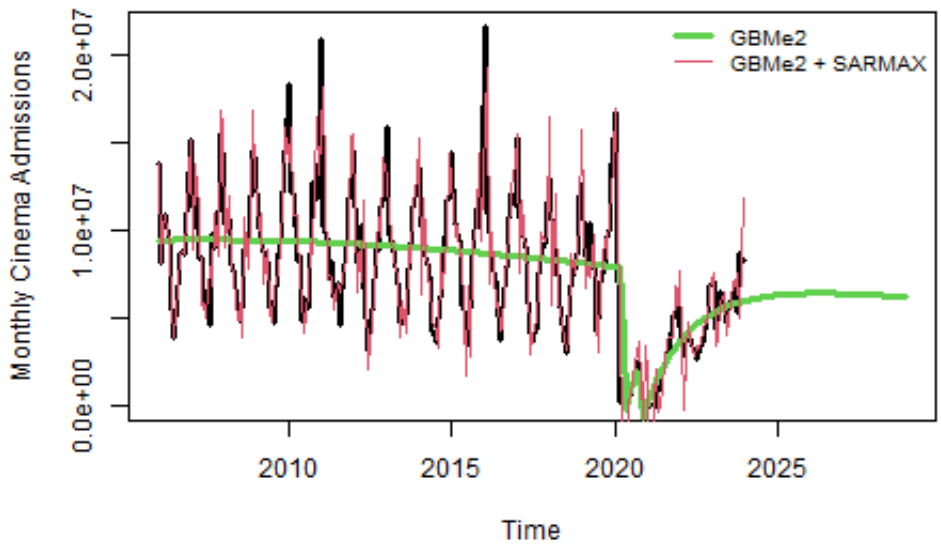
```r
model_name <- "GBMe2 + ARIMAx(2,1,0)(0,1,2)12"
aic <- sprintf("%.3f", AIC(GBM.sarmax))
dw <- NA
lj_box <- sprintf("%.3f", checkresiduals(GBM.sarmax)$p.value)
```



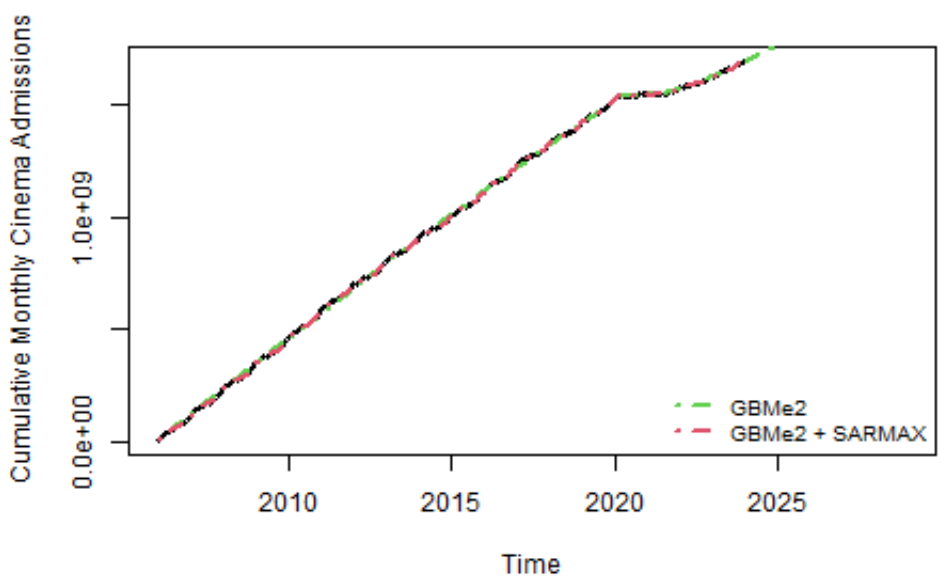

```
    Ljung-Box test

data:  Residuals from Regression with ARIMA(2,1,0)(0,1,2)[12] errors
Q* = 12.929, df = 6, p-value = 0.04417

Model df: 4.   Total lags used: 10
```

```
r2 <- sprintf("%.3f", R2_Score(as.numeric(inst_fit.GBM.sarmax), cinema_admissions))
acc <- as.data.frame.array(forecast::accuracy(GBM.sarmax))
mae <- format(acc$MAE, big.mark = ",", nsmall = 2, scientific = FALSE)
rmse <- format(acc$RMSE, big.mark = ",", nsmall = 2, scientific = FALSE)

model_performance[nrow(model_performance) + 1, ] <- c(model_name, aic, dw,
    lj_box, r2, mae, rmse)
```

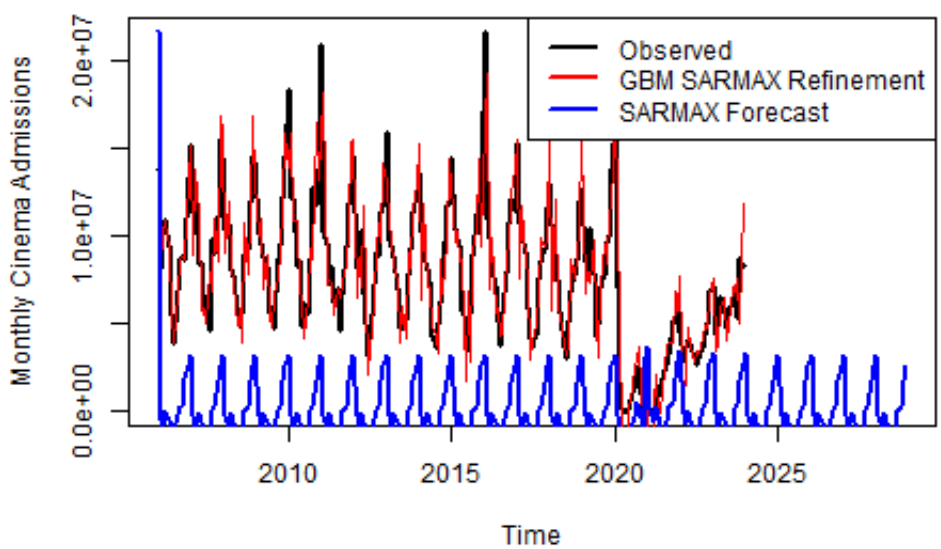
```
plot(observed_months, cinema_admissions, type = "o", pch = 16, lty = 1, cex = 0.6,
    lwd = 2, xlab = "Time", ylab = "Monthly Cinema Admissions", xlim = range(c(observed_months,
        predicted_months)), ylim = c(0, max(cinema_admissions, inst_forecast_GBM.cinema_admissions)))
lines(c(observed_months, predicted_months), inst_forecast_GBM.cinema_admissions,
    lwd = 3, col = 3)
lines(observed_months, inst_fit.GBM.sarmax, lty = 1, lwd = 1, col = 2)
legend("topright", c("GBMe2", "GBMe2 + SARMAX"), col = c(3, 2), lty = c(1,
    1), lwd = c(3, 1), merge = TRUE, cex = 0.8, bty = "n")
```




```r
plot(observed_months, cumsum(cinema_admissions), type = "b", pch = 16, lty = 3,
    cex = 0.6, lwd = 1, xlab = "Time", ylab = "Cumulative Monthly Cinema Admissions",
    xlim = range(c(observed_months, predicted_months)))
lines(c(observed_months, predicted_months), forecast_GBM.cinema_admissions,
    lty = 4, lwd = 2, col = 3)
lines(observed_months, cumsum(inst_fit.GBM.sarmax), lty = 4, lwd = 2, col = 2)
legend("bottomright", c("GBMe2", "GBMe2 + SARMAX"), col = c(3, 2), lty = c(4,
    4), lwd = c(2, 2), merge = TRUE, cex = 0.8, bty = "n")
```



```r
# Create future xreg using GBM model predictions
future.xreg <- matrix(inst_forecast_GBM.cinema_admissions, ncol = 1)

# Forecast SARMAX with xreg
GBM.sarmax_forecast <- forecast(GBM.sarmax, h = forecast_horizon, xreg = future.xreg)$mean
inst_GBM.sarmax_forecast <- make.instantaneous(GBM.sarmax_forecast)


plot(observed_months, cinema_admissions, type = "o", pch = 16, lty = 1, cex = 0.6,
    lwd = 2, xlab = "Time", ylab = "Monthly Cinema Admissions", xlim = range(c(observed_months,
        predicted_months)), ylim = c(0, max(cinema_admissions, inst_forecast_GBM.cinema_admissions)))

lines(observed_months, inst_fit.GBM.sarmax, col = "red", lwd = 1)
lines(c(observed_months, predicted_months), inst_GBM.sarmax_forecast, col = "blue",
    lwd = 2)

legend("topright", c("Observed", "GBM SARMAX Refinement", "SARMAX Forecast"),
    col = c("black", "red", "blue"), lty = c(1, 1, 1), lwd = 2)
```



!!! TODO: obviously problematic, i need another solution for the above problem.

Prophet Model

```r
# Define COVID-19 lockdown periods as holidays in Prophet
lockdowns <- data.frame(holiday = c("lockdown_1", "lockdown_2"), ds = as.Date(c("2020-03-01",
    "2020-11-01")), lower_window = c(0, 0), upper_window = c(110, 200))
holiday_season <- data.frame(holiday = "christmas_newyear", ds = as.Date(c("2006-12-01",
    "2007-12-01", "2008-12-01", "2009-12-01", "2010-12-01", "2011-12-01",
    "2012-12-01", "2013-12-01", "2014-12-01", "2015-12-01", "2016-12-01",
    "2017-12-01", "2018-12-01", "2019-12-01", "2020-12-01", "2021-12-01",
```

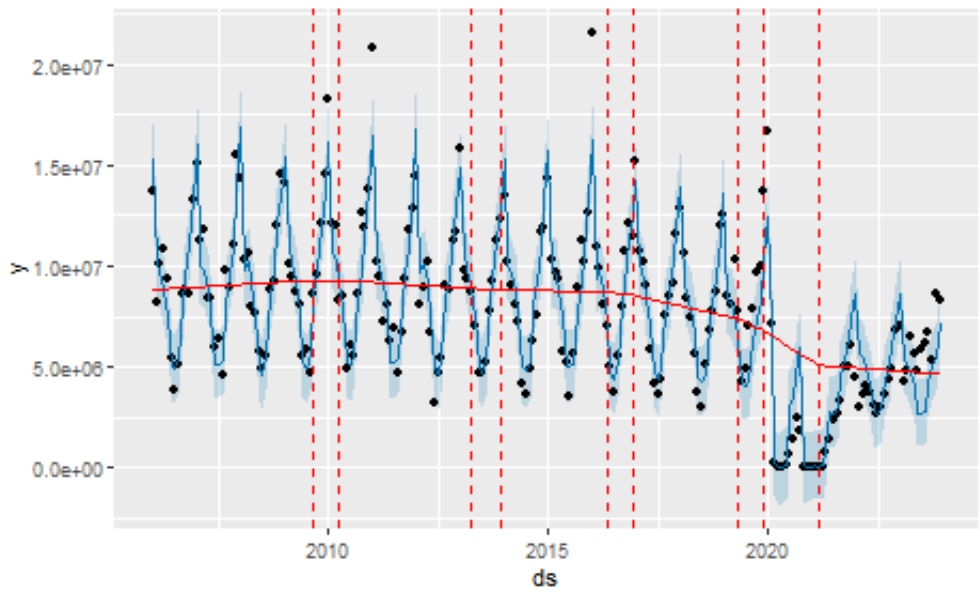
```r
    "2022-12-01", "2023-12-01", "2024-12-01", "2025-12-01", "2026-12-01",
    "2027-12-01", "2028-12-01"), lower_window = 0, upper_window = 45))
ferragosto <- data.frame(holiday = "ferragosto", ds = as.Date(c("2006-08-01",
    "2007-08-01", "2008-08-01", "2009-08-01", "2010-08-01", "2011-08-01",
    "2012-08-01", "2013-08-01", "2014-08-01", "2015-08-01", "2016-08-01",
    "2017-08-01", "2018-08-01", "2019-08-01", "2020-08-01", "2021-08-01",
    "2022-08-01", "2023-08-01", "2024-08-01", "2025-08-01", "2026-08-01",
    "2027-08-01", "2028-08-01"), lower_window = 0, upper_window = 30))

holidays <- bind_rows(lockdowns, holiday_season, ferragosto)

# Create a Date column
ott_cinema_monthly$ds <- as.Date(paste(ott_cinema_monthly$Year, rep(1:12,
    length.out = nrow(ott_cinema_monthly)), "01", sep = "-"))

# create the Prophet dataframe
prophet_data <- data.frame(ds = ott_cinema_monthly$ds, y = ott_cinema_monthly$cinema_admissions)
```

```r
# Fit Prophet model with lockdown effects
prophet_model <- prophet(holidays = holidays, yearly.seasonality = TRUE,
    seasonality.mode = "multiplicative", changepoint.prior.scale = 0.15,
    changepoint.range = 0.85)
prophet_model <- fit.prophet(prophet_model, prophet_data)

# summary(prophet_model)
```

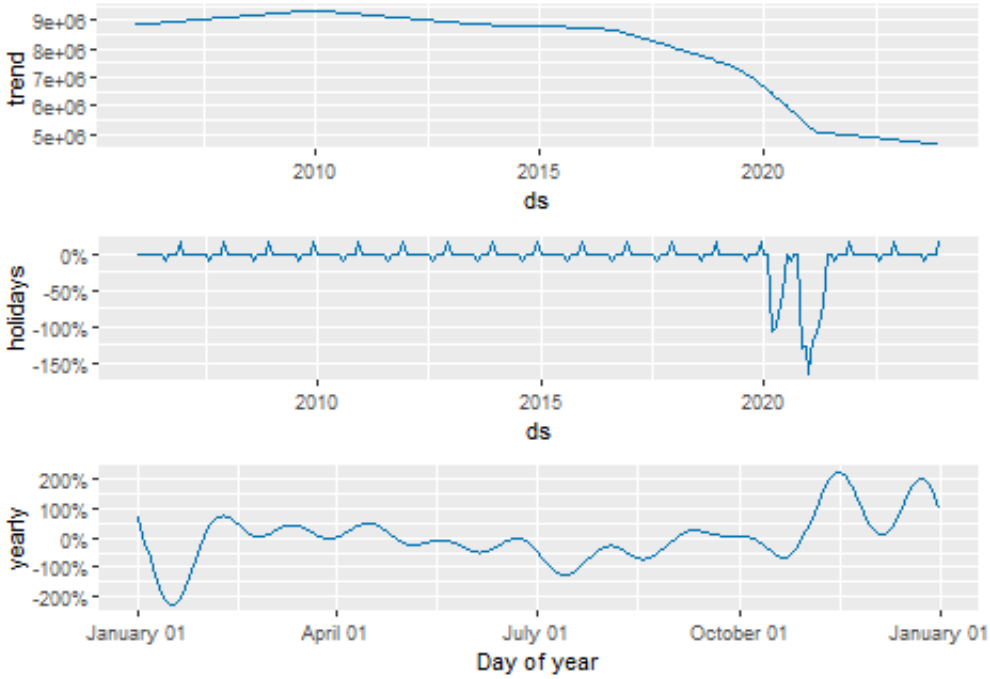
```r
# Generate future dates
future <- make_future_dataframe(prophet_model, periods = 12 * 5, freq = "month",
    include_history = TRUE)

# Predict future values
prophet_predict <- predict(prophet_model, prophet_data)
prophet_data$prophet_predict <- prophet_predict$yhat
```

```r
# dyplot.prophet(prophet_model, prophet_predict)
```

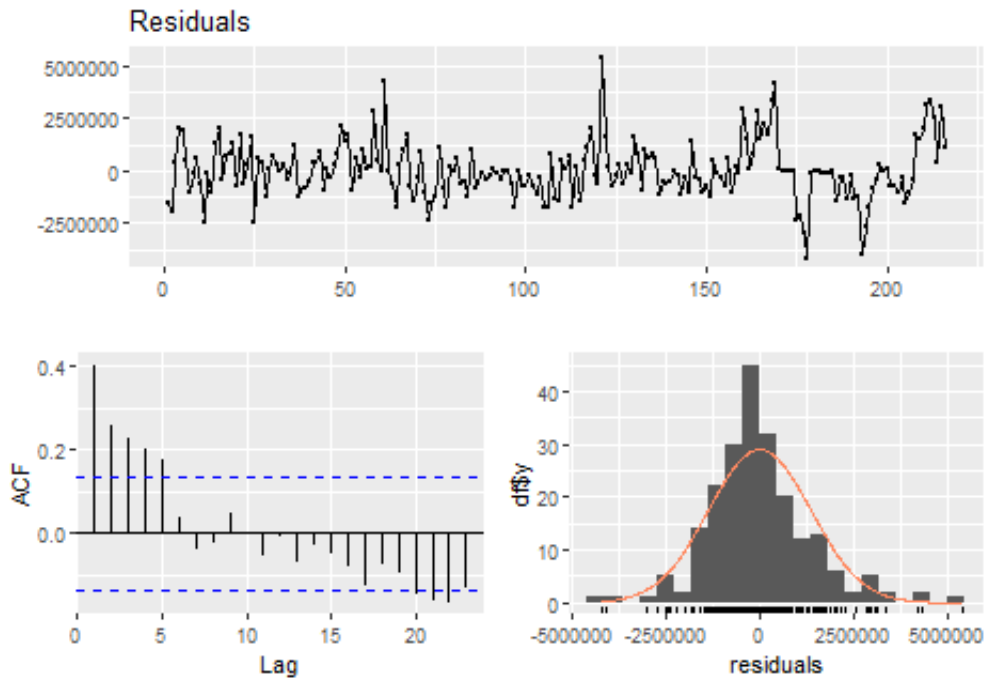
```r
# Plot the forecast with lockdown effects
plot(prophet_model, prophet_predict) + add_changepoints_to_plot(prophet_model)
```




```
prophet_plot_components(prophet_model, prophet_predict)
```

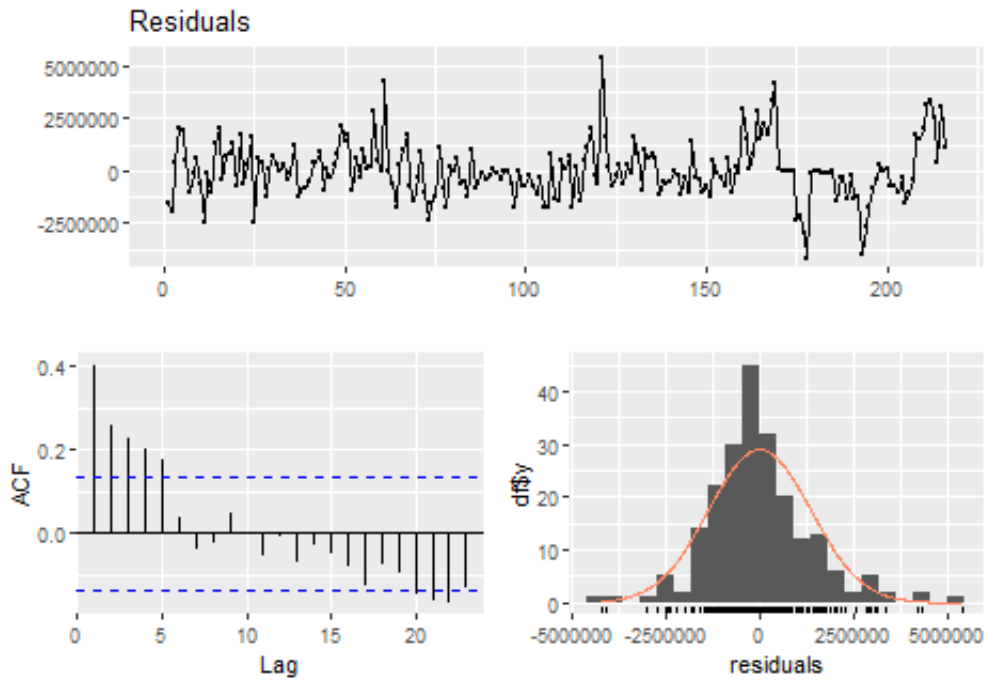



```
# Generate residuals
prophet_residuals <- prophet_data$y - prophet_data$prophet_predict
prophet_data$prophet_residuals <- prophet_residuals

checkresiduals(prophet_residuals)
```




```
    Ljung-Box test

data:  Residuals
Q* = 78.103, df = 10, p-value = 1.181e-12

Model df: 0.   Total lags used: 10
```

```
model_name <- "Prophet"
aic <- NA
dw <- NA
lj_box <- sprintf("%.3e", checkresiduals(prophet_residuals)$p.value)
```




```
    Ljung-Box test

data:  Residuals
Q* = 78.103, df = 10, p-value = 1.181e-12

Model df: 0.   Total lags used: 10
```

```
r2 <- sprintf("%.3f", R2_Score(prophet_data$prophet_predict, prophet_data$y))
mae <- format(mean(abs(prophet_residuals)), big.mark = ",", nsmall = 2, scientific = FALSE)
rmse <- format(sqrt(mean((prophet_residuals)^2)), big.mark = ",", nsmall = 2,
    scientific = FALSE)

model_performance[nrow(model_performance) + 1, ] <- c(model_name, aic, dw,
    lj_box, r2, mae, rmse)
```

Prophet with ARIMA refinement

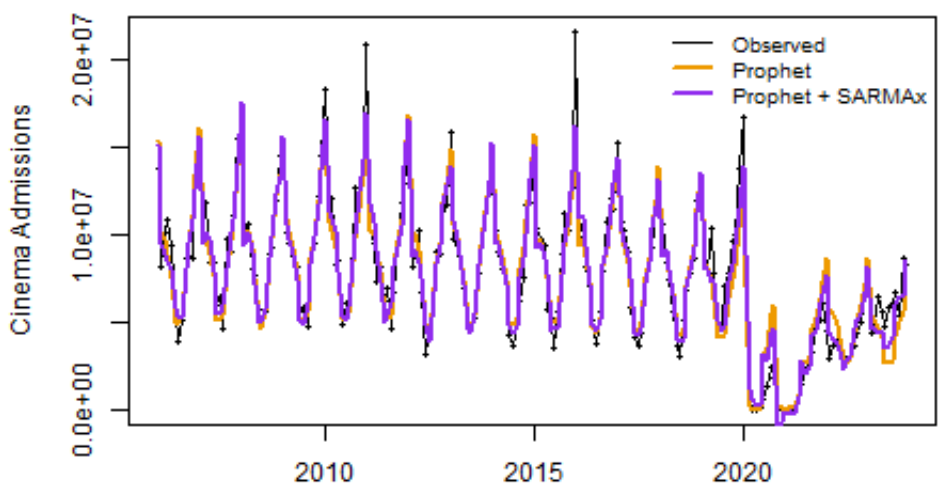
```
prop.arima <- auto.arima(cinema_admissions, xreg = prophet_data$prophet_predict,
    seasonal = TRUE)

summary(prop.arima)
```

```
Series: cinema_admissions
Regression with ARIMA(1,0,1)(2,0,0)[12] errors

Coefficients:
         ar1      ma1     sar1     sar2     xreg
```

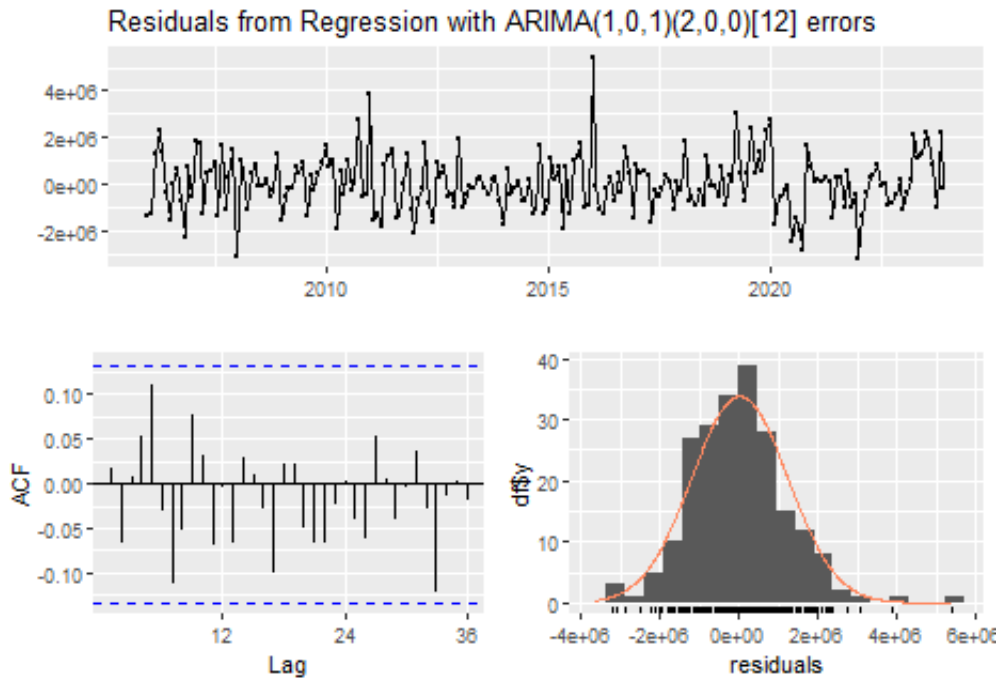
```
      0.7401  -0.4240  0.0383  -0.0928  0.9972
s.e.  0.1066   0.1435  0.0719   0.0742  0.0178

sigma^2 = 1.502e+12:  log likelihood = -3332.29
AIC=6676.57   AICc=6676.97   BIC=6696.82

Training set error measures:
                    ME    RMSE      MAE MPE MAPE      MASE       ACF1
Training set 22007.65 1211338 929898.9 NaN  Inf 0.5231961 0.01639939
```

```r
fit.prop.arima <- fitted(prop.arima)
prophet_data$combined_predict <- fit.prop.arima
```

```r
plot(ott_cinema_monthly$ds, ott_cinema_monthly$cinema_admissions, type = "o",
    pch = 16, cex = 0.6, xlab = "", ylab = "Cinema Admissions")

lines(ott_cinema_monthly$ds, prophet_data$prophet_predict, col = "orange2",
    lwd = 2)
lines(ott_cinema_monthly$ds, as.numeric(fit.prop.arima), col = "purple2",
    lwd = 2)

legend("topright", c("Observed", "Prophet", "Prophet + SARMAx"), col = c("black",
    "orange2", "purple2"), lwd = c(1, 2, 2), lty = 1, bty = "n", cex = 0.85)
```




```r
model_name <- "Prophet + SARMAx"
aic <- sprintf("%.3f", AIC(prop.arima))
dw <- NA
lj_box <- sprintf("%.3f", checkresiduals(prop.arima)$p.value)
```




```
    Ljung-Box test

data:  Residuals from Regression with ARIMA(1,0,1)(2,0,0)[12] errors
Q* = 17.857, df = 20, p-value = 0.5968

Model df: 4.   Total lags used: 24
```

```
r2 <- sprintf("%.3f", R2_Score(prophet_data$combined_predict, prophet_data$y))
acc <- as.data.frame.array(forecast::accuracy(prop.arima))
mae <- format(acc$MAE, big.mark = ",", nsmall = 2, scientific = FALSE)
rmse <- format(acc$RMSE, big.mark = ",", nsmall = 2, scientific = FALSE)

model_performance[nrow(model_performance) + 1, ] <- c(model_name, aic, dw,
    lj_box, r2, mae, rmse)
```

Exponential Smoothing

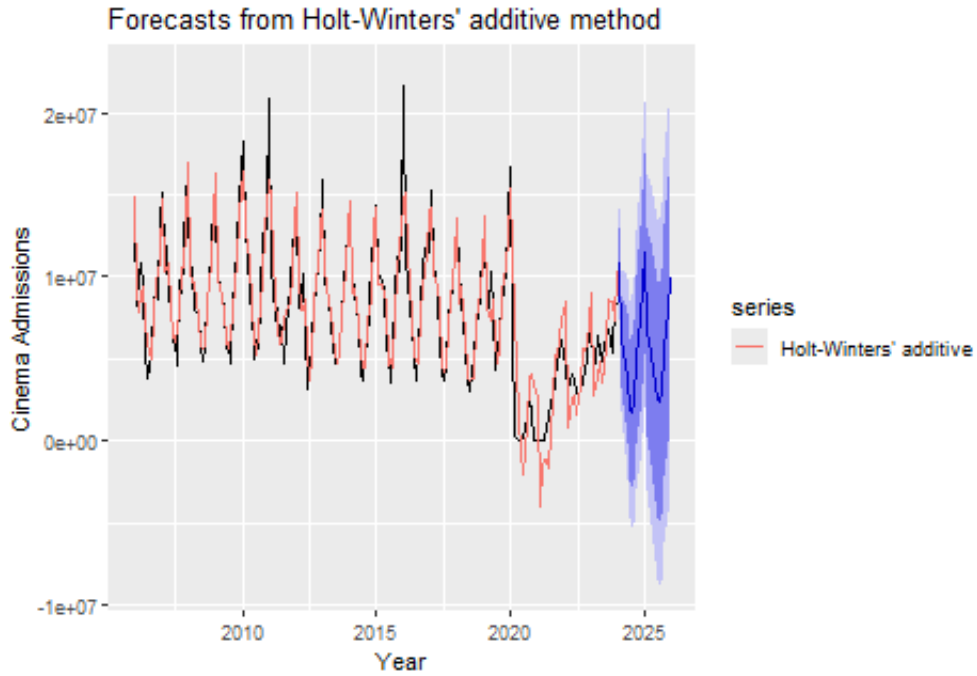
Holt-Winters method (for Trend and Seasonality)

```
hw.cinema_admissions <- hw(cinema_admissions, seasonal = "additive")


summary(hw.cinema_admissions)
```

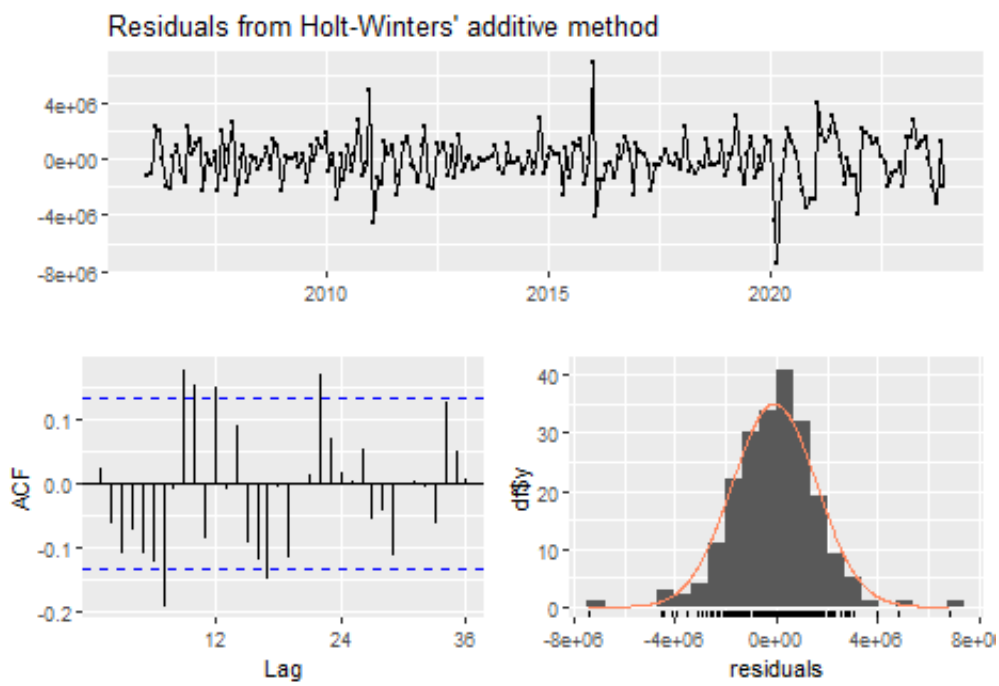
```
Forecast method: Holt-Winters' additive method
```

```
Model Information:
Holt-Winters' additive method

Call:
hw(y = cinema_admissions, seasonal = "additive")

  Smoothing parameters:
    alpha = 0.7404
    beta  = 1e-04
    gamma = 1e-04

  Initial states:
    l = 9169380.6501
    b = 49001.19
    s = 3696598 1749141 634656.7 -597404.5 -3094459 -3658436
           -3482387 -1529110 -532336.9 211025.5 903663 5699050

  sigma:  1733522

     AIC     AICc      BIC
7384.404 7387.495 7441.784

Error measures:
                    ME    RMSE     MAE MPE MAPE      MASE      ACF1
Training set -90448.17 1668083 1248623 NaN  Inf 0.7025221 0.0226788

Forecasts:
         Point Forecast      Lo 80    Hi 80      Lo 95    Hi 95
Jan 2024       10844047  8622448.9 13065645  7446405.9 14241688
Feb 2024        6095348  3330940.4  8859757  1867551.2 10323146
Mar 2024        5450186  2233178.8  8667194   530198.0 10370174
Apr 2024        4753858  1140404.2  8367313  -772442.9 10280160
May 2024        3803793  -166811.4  7774397 -2268722.3  9876308
Jun 2024        1897661 -2400603.0  6195926 -4675966.7  8471289
Jul 2024        1768992 -2833745.0  6371729 -5270286.7  8808270
Aug 2024        2379760 -2508596.7  7268116 -5096336.2  9855855
Sep 2024        4923637  -234618.3 10081893 -2965233.8 12812508
Oct 2024        6202615   787829.4 11617401 -2078585.0 14483815
Nov 2024        7364592  1704827.4 13024357 -1291271.3 16020456
Dec 2024        9358782  3464148.5 15253416   343717.9 18373847
Jan 2025       11408617  5288000.5 17529233  2047942.1 20769291
Feb 2025        6659918   321371.4 12998465 -3034052.5 16353888
Mar 2025        6014756  -534529.6 12564041 -4001511.8 16031023
Apr 2025        5318428 -1435077.3 12071933 -5010167.1 15647023
May 2025        4368362 -2583418.9 11320144 -6263469.6 15000194
Jun 2025        2462231 -4682377.2  9606839 -8464504.3 13388966
Jul 2025        2333561 -4998854.4  9665977 -8880400.8 13547523
Aug 2025        2944329 -4571251.1 10459909 -8549759.0 14438417
Sep 2025        5488207 -2206226.8 13182640 -6279413.8 17255827
Oct 2025        6767185 -1102084.7 14636454 -5267824.2 18802193
Nov 2025        7929162  -111187.9 15969512 -4367492.0 20225816
Dec 2025        9923352  1715442.1 18131262 -2629562.9 22476267
```

```r
autoplot(hw.cinema_admissions) + autolayer(fitted(hw.cinema_admissions),
    series = "Holt-Winters' additive") + ylab("Cinema Admissions") + xlab("Year")
```




```r
model_name <- "Holt-Winters Exponential"
aic <- sprintf("%.3f", hw.cinema_admissions$model$aic)
dw <- NA
lj_box <- sprintf("%.3e", checkresiduals(hw.cinema_admissions)$p.value)
```



```
    Ljung-Box test

data:  Residuals from Holt-Winters' additive method
Q* = 63.089, df = 24, p-value = 2.311e-05

Model df: 0.   Total lags used: 24
```

```r
r2 <- sprintf("%.3f", R2_Score(hw.cinema_admissions$fitted, cinema_admissions))
acc <- as.data.frame.array(forecast::accuracy(hw.cinema_admissions))
mae <- format(acc$MAE, big.mark = ",", nsmall = 2, scientific = FALSE)
rmse <- format(acc$RMSE, big.mark = ",", nsmall = 2, scientific = FALSE)

model_performance[nrow(model_performance) + 1, ] <- c(model_name, aic, dw,
    lj_box, r2, mae, rmse)
```

```r
kable(as.data.frame(model_performance))
```

Model	AIC	DW_Test	Ljung_Box_pval	R2	MAE	RMSE
TSLM - Full	6054.269	1.065	NA	0.995	193,057.61	267,895.01
TSLM - Stepwise	6050.035	1.064	NA	0.995	195,166.90	268,992.46
ARIMA(2,1,5)(0,1,1)[12]	6422.800	NA	0.160	0.833	1,171,316.97	1,621,661.60
auto.ARIMA(1,0,2)(0,1,1)[12]	6448.711	NA	0.108	0.828	1,158,030.22	1,647,445.01
auto.ARIMAx(3,1,0)(2,0,1)[12]	5974.656	NA	0.047	0.996	170,544.06	245,186.45
ARIMAx(1,1,2)(0,1,1)[12]	5632.803	NA	0.782	0.997	159,146.56	233,990.99
Bass Model	NA	NA	< 2.2e-16	1.000	2,772,250.27	3,480,549.43
GBMe2	NA	NA	< 2.2e-16	1.000	2,307,165.36	3,106,753.77
GBMe2 + ARIMAx(2,1,0)(0,1,2)12	6379.206	NA	0.044	0.718	1,061,865.94	1,490,790.20
Prophet	NA	NA	1.181e-12	0.885	975,508.94	1,348,348.68
Prophet + SARMAx	6676.570	NA	0.597	0.907	929,898.93	1,211,338.20
Holt-Winters Exponential	7384.404	NA	2.311e-05	0.824	1,248,622.75	1,668,082.59

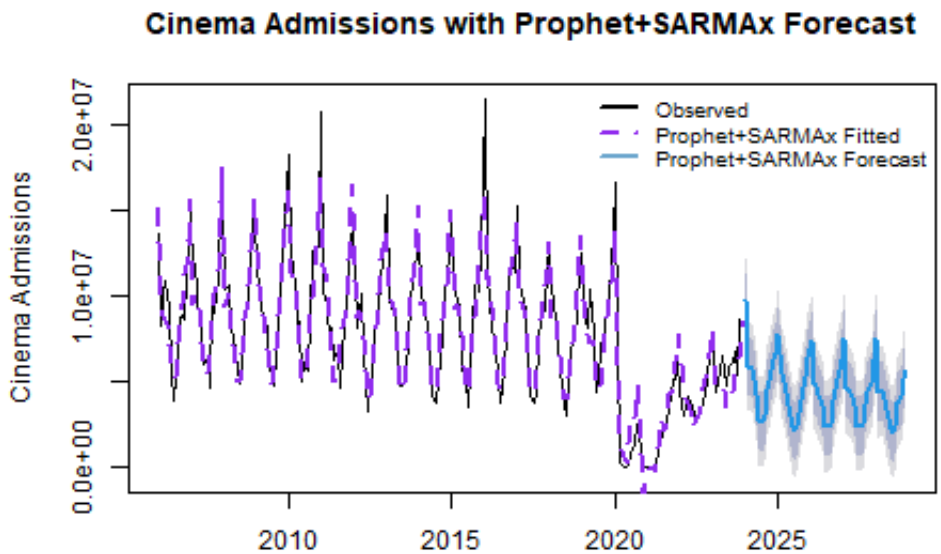
3) Forecasting

Forecasting with Prophet + ARIMA

```r
# Create 12-month future dataframe
future <- make_future_dataframe(prophet_model, periods = forecast_horizon,
    freq = "month", include_history = FALSE)
prophet_forecast <- predict(prophet_model, future)
```

```r
# Forecast residuals using ARIMA
prop.arima_forecast <- forecast(prop.arima, h = forecast_horizon, xreg = prophet_forecast$yhat)
```

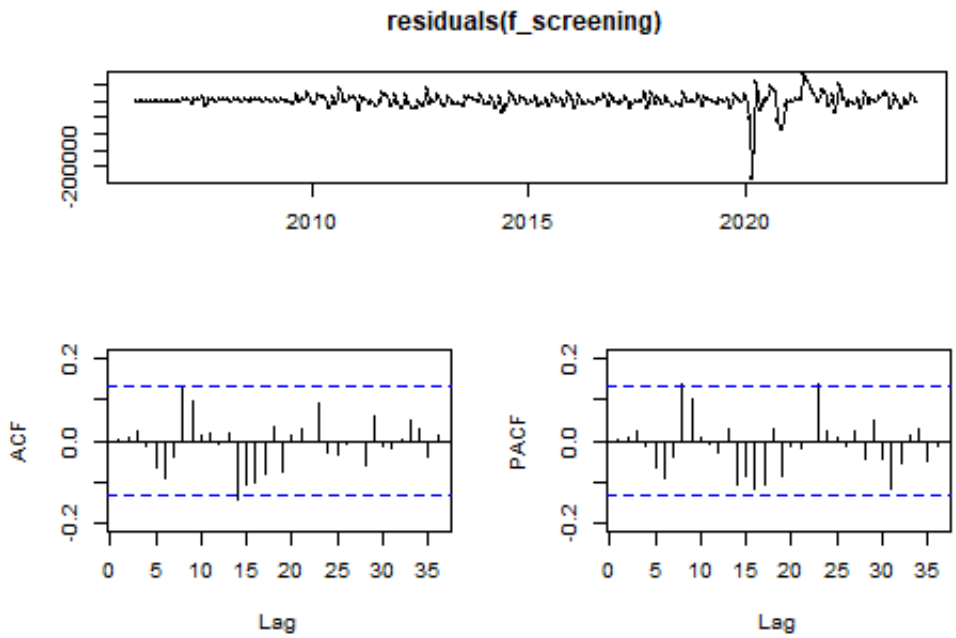
```r
plot(prop.arima_forecast, xlab = "", ylab = "Cinema Admissions", main = "Cinema Admissions with Prophet+
lines(fit.prop.arima, col = "purple2", lwd = 2, lty = 2)

# Add legend
legend("topright", legend = c("Observed", "Prophet+SARMAx Fitted", "Prophet+SARMAx Forecast"),
    col = c("black", "purple2", "skyblue3"), cex = 0.8, lty = c(1, 2, 1),
    lwd = c(2, 2, 2), pch = c(NA, NA, NA), pt.cex = 0.5, bty = "n")
```



Forecast with ARIMAx

```
horizon <- 12 * 5
# Forecast each regressor

f_screening <- forecast(Arima(cinema_screening, order = c(2, 1, 5), seasonal = c(0,
    1, 1)), h = horizon)
f_spending <- forecast(Arima(cinema_spending, order = c(2, 1, 6), seasonal = c(0,
    1, 1)), h = horizon)
f_avg_spending <- forecast(Arima(avg_cinema_spending_pp, order = c(2, 1,
    3), seasonal = c(1, 1, 0)), h = horizon)
f_cpi <- forecast(Arima(cpi_all, order = c(3, 1, 3), seasonal = c(1, 1, 1),
    include.drift = TRUE), h = horizon)

# Build future xreg matrix
xreg1_future <- cbind(screening = as.numeric(f_screening$mean), spending = as.numeric(f_spending$mean),
    avg_spending = as.numeric(f_avg_spending$mean), cpi = as.numeric(f_cpi$mean))
```

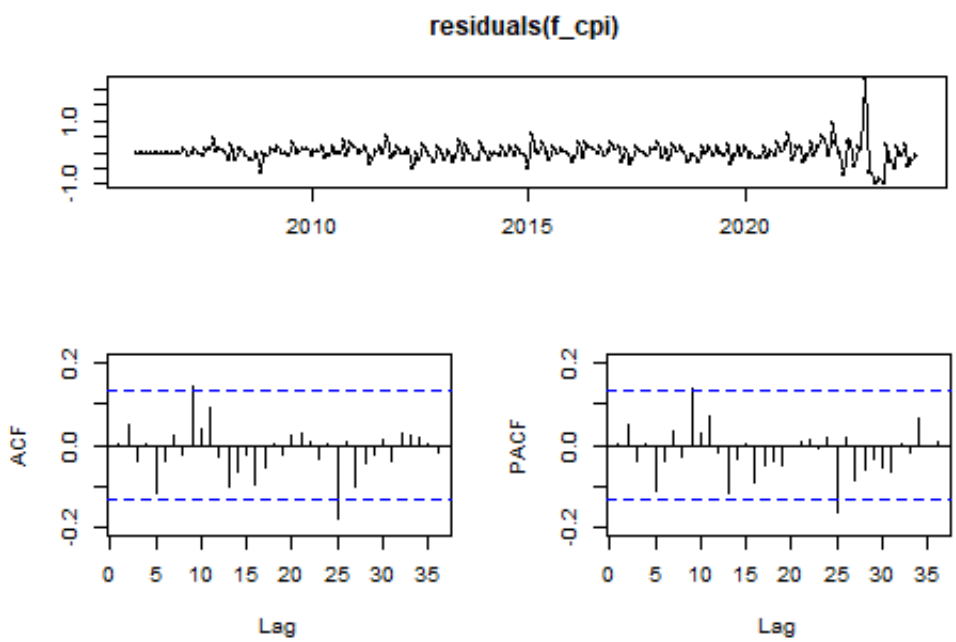
```
tsdisplay(residuals(f_screening))
```




```
tsdisplay(residuals(f_spending))
```

```
tsdisplay(residuals(f_avg_spending))
```

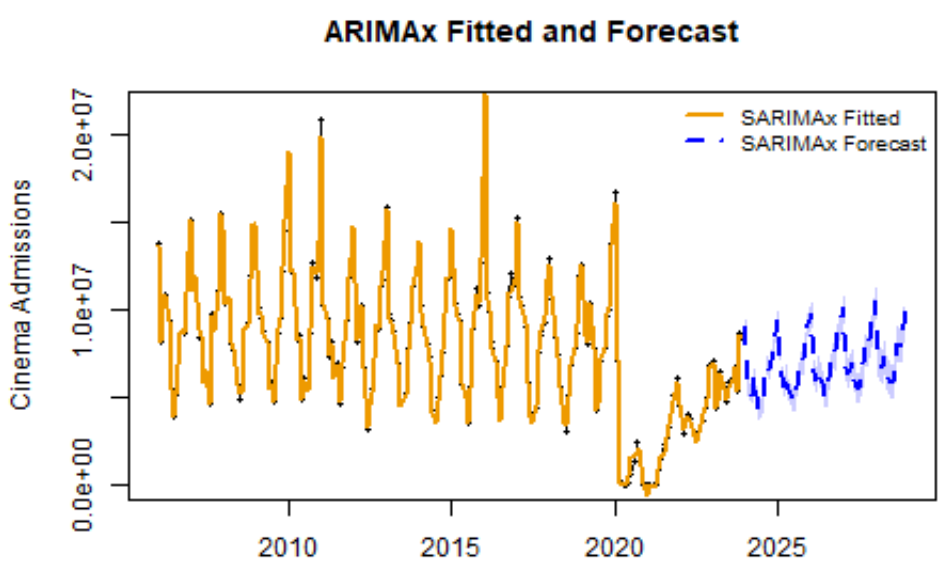
```r
tsdisplay(residuals(f_cpi))
```




```r
sarimax_forecast <- forecast(sarimax_model2, h = horizon, xreg = xreg1_future)

# Extract components
sarimax_mean <- sarimax_forecast$mean
sarimax_upper <- sarimax_forecast$upper[, 2]
sarimax_lower <- sarimax_forecast$lower[, 2]
```

```r
forecast_dates <- seq(from = as.Date("2024-01-01"), by = "month", length.out = horizon)
```

```r
# Combine observed and forecast dates
full_dates <- c(ott_cinema_monthly$ds, forecast_dates)

# Compute y-axis range to include forecast values
y_range <- range(c(cinema_admissions, sarimax_mean, sarimax_upper, sarimax_lower),
    na.rm = TRUE)

# Base plot with actual data and fitted values
plot(ott_cinema_monthly$ds, ott_cinema_monthly$cinema_admissions, type = "o",
    pch = 16, cex = 0.6, xlab = "", ylab = "Cinema Admissions", main = "ARIMAx Fitted and Forecast",
    xlim = range(full_dates), ylim = y_range)

lines(ott_cinema_monthly$ds, fit.sarimax2, col = "orange2", lwd = 2)

# Add forecast confidence interval (shaded)
forecast_dates <- seq(from = as.Date("2024-01-01"), by = "month", length.out = horizon)
polygon(c(forecast_dates, rev(forecast_dates)), c(sarimax_upper, rev(sarimax_lower)),
    col = rgb(0, 0, 1, 0.2), border = NA)

# Add forecast line
lines(forecast_dates, sarimax_mean, col = "blue", lwd = 2, lty = 2)

# Add legend
legend("topright", legend = c("SARIMAx Fitted", "SARIMAx Forecast"), col = c("orange2",
    "blue"), cex = 0.8, lty = c(1, 2), lwd = c(2, 2), pch = c(NA, NA), pt.cex = 2,
    bty = "n")
```

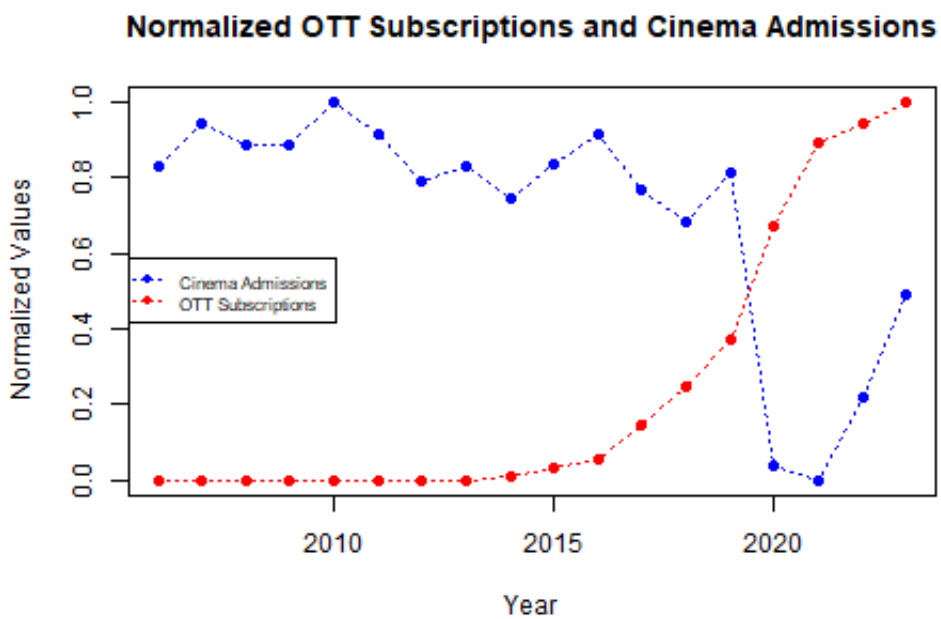


4) Unbalanced Competition Regime Change Diachronic model (UCRCD)

Perform a competition study between OTT and Cinema

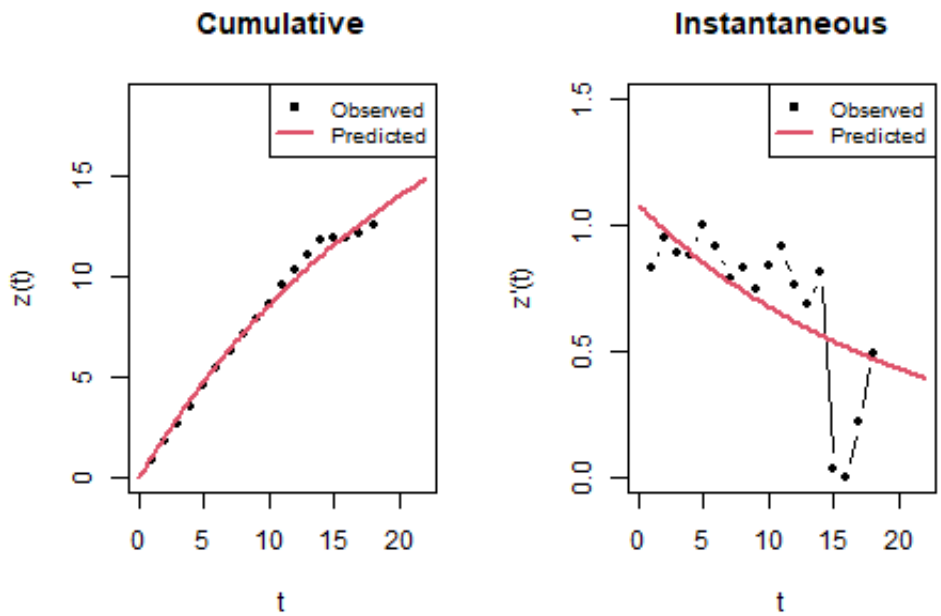
```
# Apply Min-Max Normalization
ott_subscriptions.norm <- min_max_scale(ott_cinema_annual$ott_subscriptions_M)
cinema_admissions.norm <- min_max_scale(ott_cinema_annual$cinema_admissions)
```

```
# Plot the normalized time series
plot(ott_cinema_annual$Year, cinema_admissions.norm, type = "o", pch = 16,
    lty = 3, col = "blue", xlab = "Year", ylab = "Normalized Values", main = "Normalized OTT Subscripti
    ylim = c(0, 1))
points(ott_cinema_annual$Year, ott_subscriptions.norm, type = "o", pch = 16,
    lty = 3, col = "red")
legend("left", legend = c("Cinema Admissions", "OTT Subscriptions"), col = c("blue",
    "red"), lty = 3, pch = 16, cex = 0.7)
```

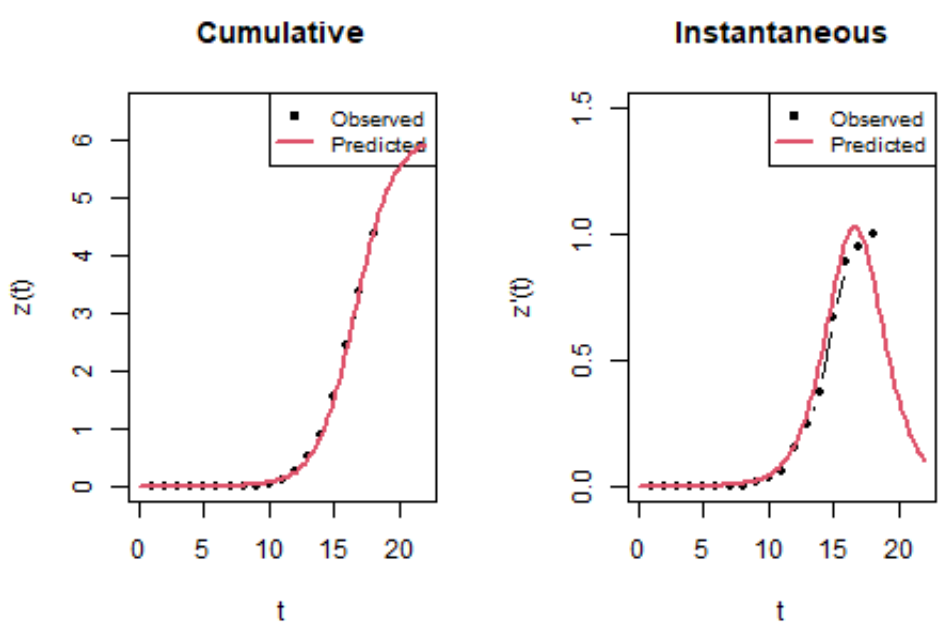


Estimating initial parameters with Bass Model:

```
# Cinema Admissions
BM.cinema_admissions.norm <- BM(cinema_admissions.norm, display = T)
```



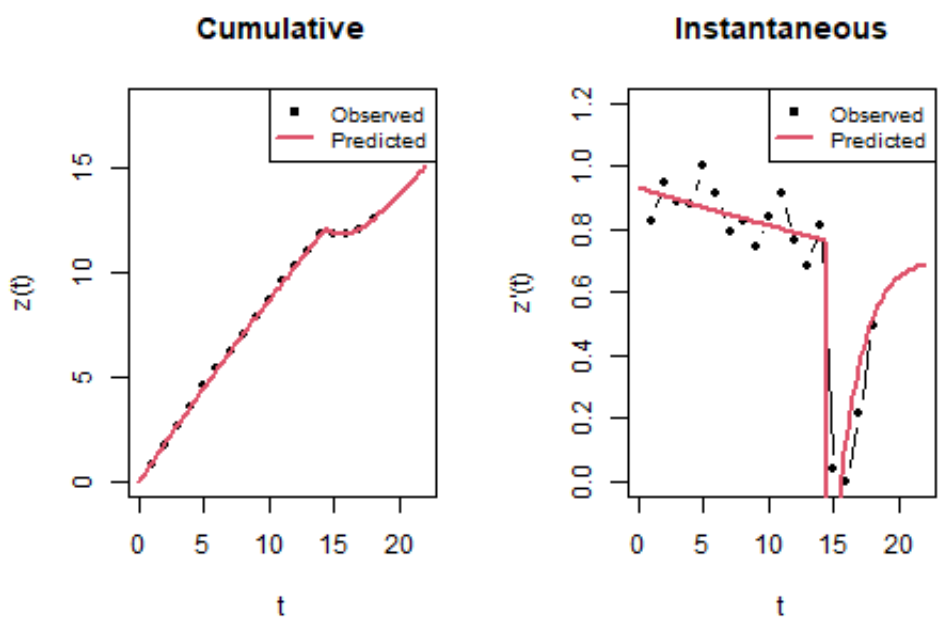

```
m.c = BM.cinema_admissions.norm$Estimate[1, 1]  # Market potential for Cinema
p.c = BM.cinema_admissions.norm$Estimate[2, 1]  # Innovation coefficient for Cinema
q.c = BM.cinema_admissions.norm$Estimate[3, 1]  # Imitation coefficient for Cinema

# OTT Subscriptions
BM.ott_subscriptions.norm <- BM(ott_subscriptions.norm, display = T)
```



```r
m.o = BM.ott_subscriptions.norm$Estimate[1, 1]  # Market potential for OTT
p.o = BM.ott_subscriptions.norm$Estimate[2, 1]  # Innovation coefficient for OTT
q.o = BM.ott_subscriptions.norm$Estimate[3, 1]  # Imitation coefficient for OTT
```

GBM for cinema admissions:

```r
# Cinema Admissions GBM exponential shock

# Define the time period when the COVID-19 shock begins
covid_start <- 2020 - 2006 + 1

# m,p,q, a(starting point), b(memory of the shock), c(intensity)
GBM.cinema_admissions.norm <- GBM(cinema_admissions.norm, shock = "exp",
    nshock = 1, prelimestimates = c(m.c, p.c, q.c, covid_start, -0.5, -0.1))
```

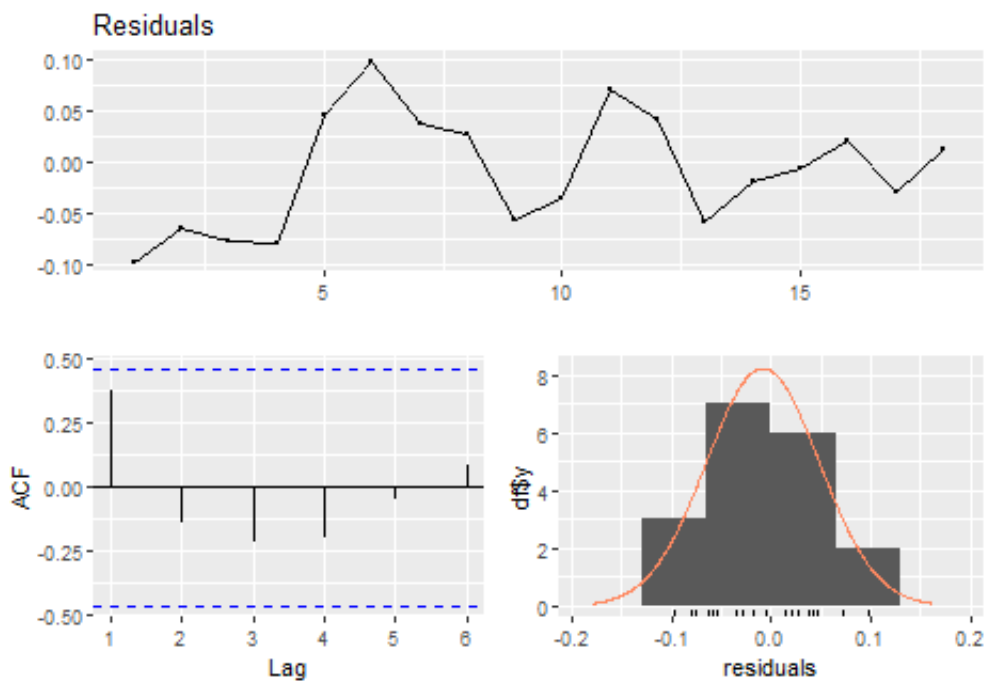



```r
summary(GBM.cinema_admissions.norm)
```

```
Call: ( Generalized Bass model with 1  Exponential  shock )

  GBM(series = cinema_admissions.norm, shock = "exp", nshock = 1,
    prelimestimates = c(m.c, p.c, q.c, covid_start, -0.5, -0.1))

Residuals:
      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
-0.096358 -0.057368 -0.011294 -0.008509  0.035628  0.098147

Coefficients:
           Estimate    Std.Error         Lower         Upper  p-value
m    581.878081928 3.033552e+04 -5.887465e+04 60038.4067906 9.85e-01
```

```
p      0.001599336 8.411980e-02 -1.632724e-01     0.1664711 9.85e-01
q     -0.012431924 7.304014e-02 -1.555880e-01     0.1307241 8.68e-01
a1    14.357728896 1.620019e-01  1.404021e+01    14.6752467 2.84e-18 ***
b1    -0.464933344 1.442744e-01 -7.477059e-01    -0.1821608 7.32e-03  **
c1    -1.743752639 3.255658e-01 -2.381850e+00    -1.1056555 1.72e-04 ***
---
 Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

 Residual standard error  0.068219  on  12  degrees of freedom
 Multiple R-squared:   0.999785  Residual sum of squares:  0.055846
```

```r
checkresiduals(GBM.cinema_admissions.norm)
```

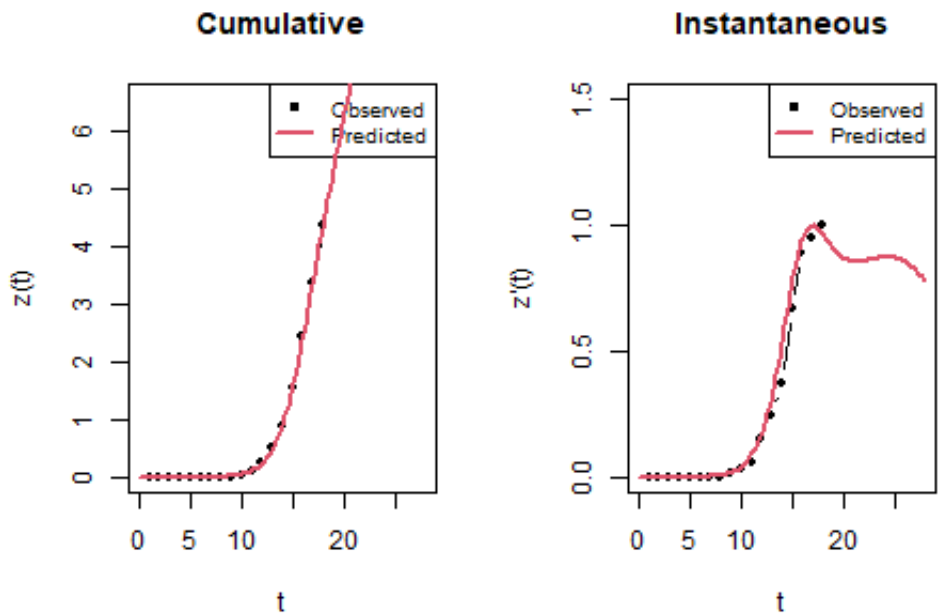



```
    Ljung-Box test

data:  Residuals
Q* = 5.5125, df = 4, p-value = 0.2386

Model df: 0.   Total lags used: 4
```

```r
m.c = BM.cinema_admissions.norm$Estimate[1, 1]  # Market potential for Cinema
p.c = BM.cinema_admissions.norm$Estimate[2, 1]  # Innovation coefficient for Cinema
q.c = BM.cinema_admissions.norm$Estimate[3, 1]  # Imitation coefficient for Cinema
```

GGM for OTT subscriptions:

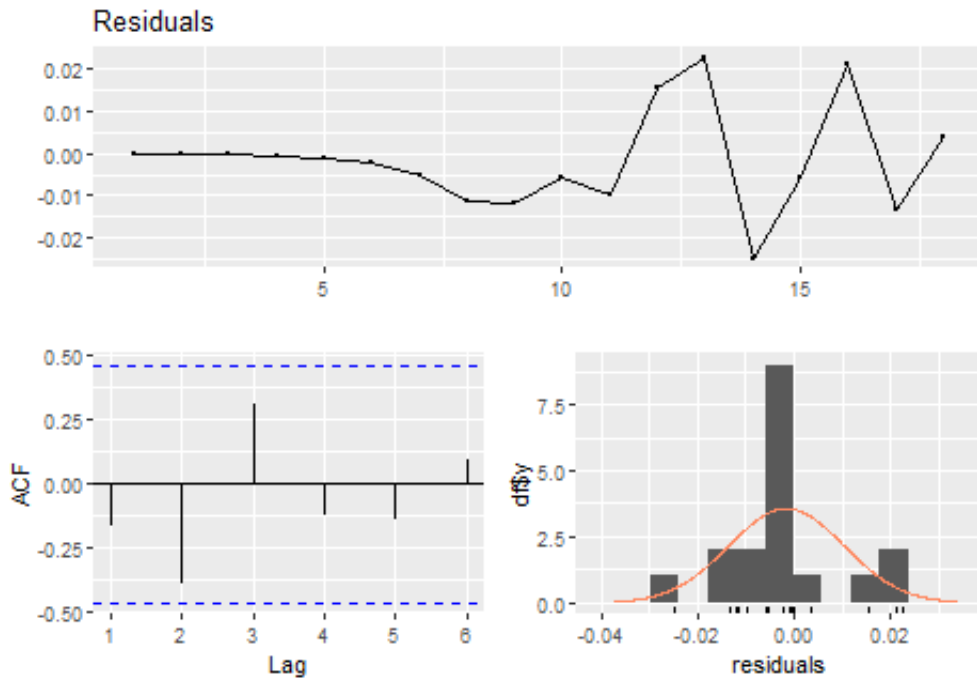
```r
GGM.ott_subscriptions.norm <- GGM(ott_subscriptions.norm, prelimestimates = c(m.o,
    0.01, 0.01, p.o, q.o), oos = 10)
```




```r
summary(GGM.ott_subscriptions.norm)
```

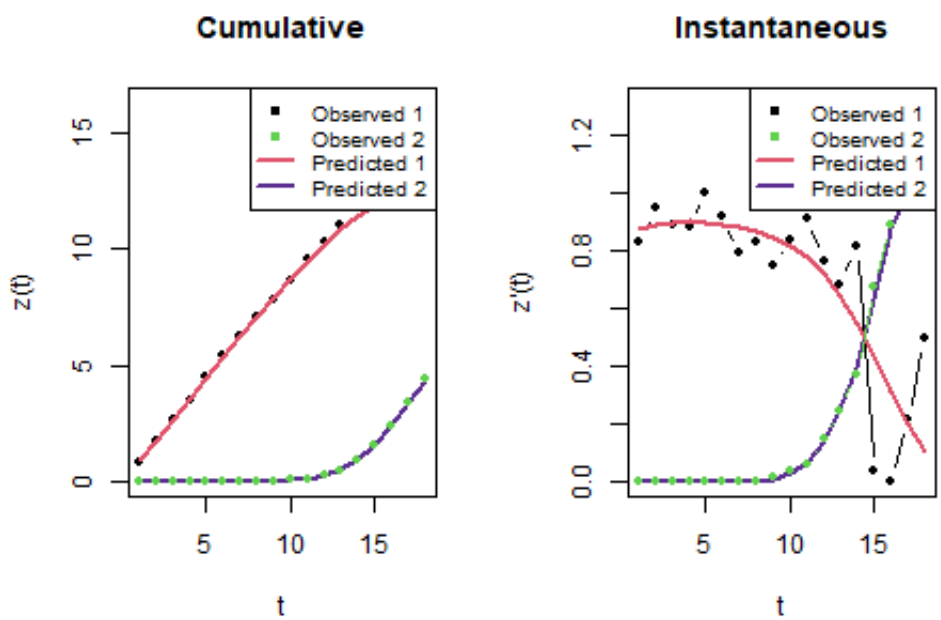
```
Call: ( Guseo Guidolin Model )

  GGM(series = ott_subscriptions.norm, prelimestimates = c(m.o,
    0.01, 0.01, p.o, q.o), oos = 10)

Residuals:
       Min.    1st Qu.     Median       Mean    3rd Qu.       Max.
-2.478e-02 -8.823e-03 -1.650e-03 -1.613e-03 -2.339e-05  2.257e-02

Coefficients:
          Estimate    Std.Error         Lower        Upper  p-value
K    1.822034e+01 1.813083e+02 -3.371374e+02 3.735781e+02 9.21e-01
pc   2.753476e-04 4.188104e-03 -7.933185e-03 8.483880e-03 9.49e-01
qc   2.436747e-01 4.896156e-01 -7.159542e-01 1.203304e+00 6.27e-01
ps   2.795305e-05 5.303582e-05 -7.599524e-05 1.319013e-04 6.07e-01
qs   6.486660e-01 1.148359e-01  4.235917e-01 8.737403e-01 7.95e-05 ***
---
 Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

 Residual standard error  0.013758  on  13  degrees of freedom
 Multiple R-squared:   0.999917  Residual sum of squares:  0.002461
```

```r
checkresiduals(GGM.ott_subscriptions.norm)
```




```
    Ljung-Box test

data:  Residuals
Q* = 6.655, df = 4, p-value = 0.1553

Model df: 0.   Total lags used: 4
```

```
###estimate the UCRCD (with delta and gamma)
UCRCD.adm.subs.norm<- UCRCD(
  series1 = cinema_admissions.norm,  # Instantaneous cinema admissions
  series2 = ott_subscriptions.norm,  # Instantaneous OTT subscriptions
  m1 = m.c,                          # Market potential for cinema (GBM result)
  p1c = p.c,                         # Innovation coefficient for cinema
  q1c = q.c,                         # Imitation coefficient for cinema
  m2 = m.o,                          # Market potential for OTT (BM result)
  p2 = p.o,                         # Innovation coefficient for OTT
  q2 = q.o                          # Imitation coefficient for OTT
)
```




```r
summary(UCRCD.adm.subs.norm)
```

```
Call: ( UCRCD Model )

  UCRCD(series1 = cinema_admissions.norm, series2 = ott_subscriptions.norm,
    m1 = m.c, m2 = m.o, p1c = p.c, q1c = q.c, p2 = p.o, q2 = q.o)

Residuals Series 1:
      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
-0.400803 -0.045457  0.020390  0.005014  0.053529  0.392080

Residuals Series 2:
       Min.    1st Qu.     Median       Mean    3rd Qu.       Max.
-6.585e-02 -1.153e-03 -3.615e-05  2.162e-04  2.489e-03  4.263e-02

Coefficients:
            Estimate   Std.Error        Lower       Upper  p-value
mc     2.113772e+01 1.858348987 17.495421293 24.78001546 3.29e-12 ***
p1c    4.068835e-02 0.005394613  0.030115108  0.05126160 2.58e-08 ***
p2    -6.260099e-05 0.004807900 -0.009485912  0.00936071 9.90e-01
q1c   -2.589281e-01 0.105988731 -0.466662199 -0.05119401 2.09e-02   *
q2     1.110295e+00 0.310919900  0.500902930  1.71968654 1.26e-03  **
delta  3.210642e-01 0.114933126  0.095799415  0.54632899 9.14e-03  **
gamma  1.109853e+00 0.320895724  0.480908569  1.73879669 1.70e-03  **
---
 Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

 Residual standard error Series 1: 0.222172  on  11  degrees of freedom
 Residual standard error Series 2: 0.027163  on  11  degrees of freedom
 Multiple R-squared:   0.904185  Residual sum of squares:  0.55108
```